
Курс: Рекомендательные системы

Занятие 18:

Ранжирование, CTR Prediction, нейросетевое ранжирование

Лашинин Олег

27.05.26

Почему сегодня эта тема?

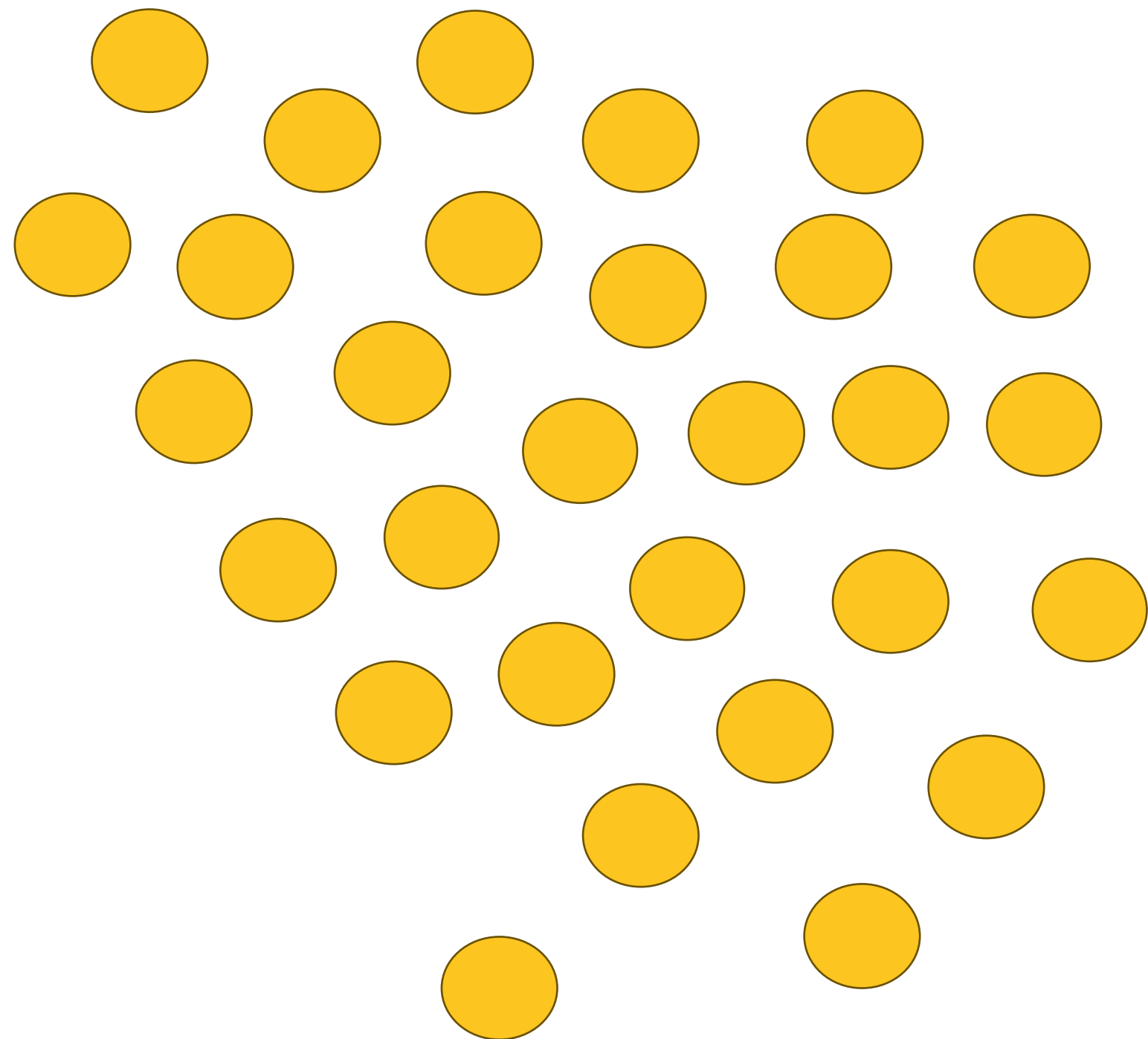
Ранжирование является важной частью хороших
рекомендательных пайплайнов

План занятия



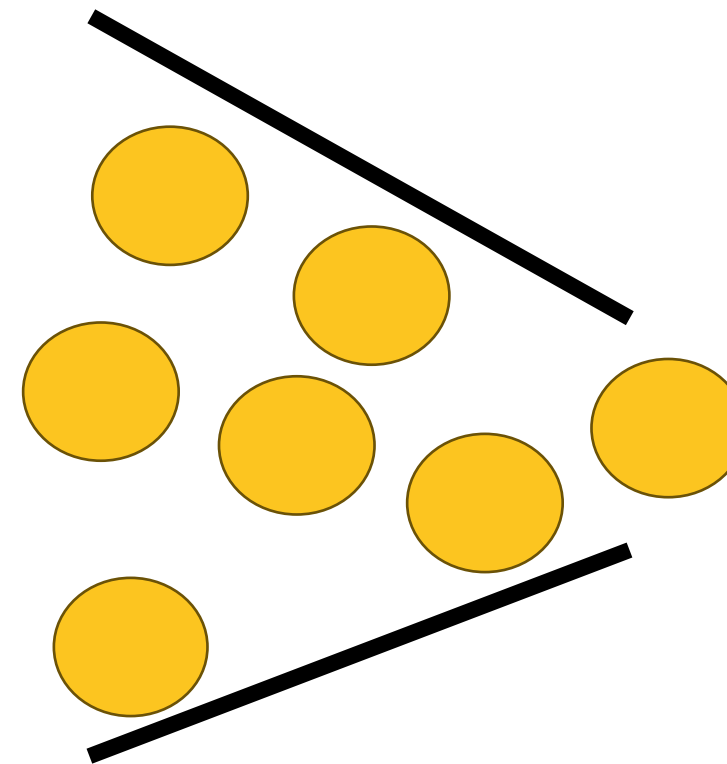
- Про ранжирование
- Не нейросетевое ранжирование
- Нейросетевое ранжирование

Идея двухуровневой системы



Очень много айтемов
(кандидатогенерация)

Модель выбора



	Item Features	User Features	1
	Item Features	User Features	1
	Item Features	User Features	0
	Item Features	User Features	0

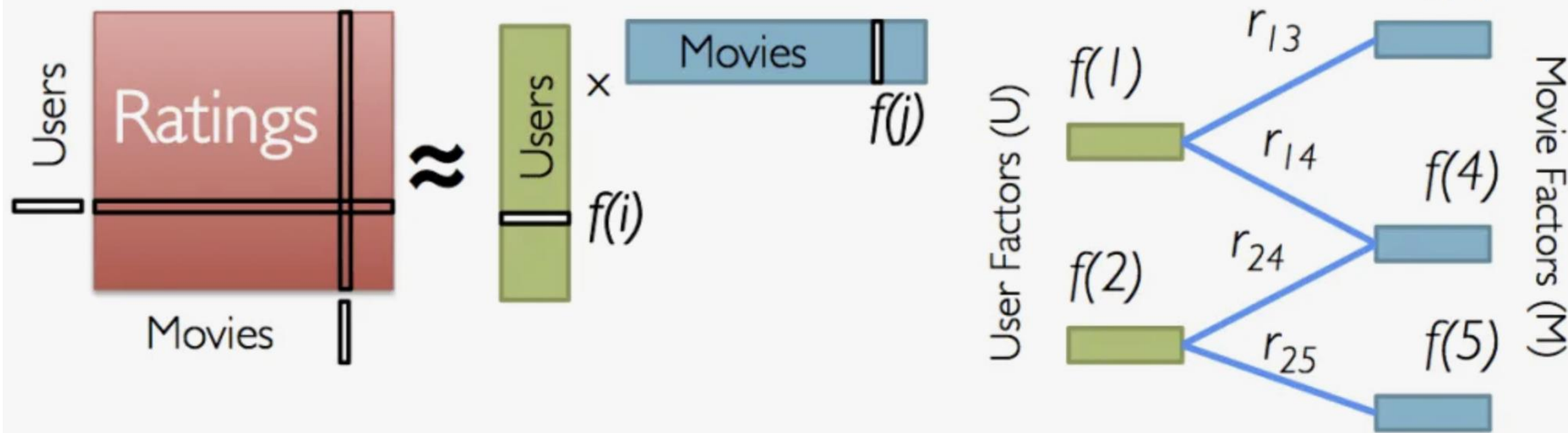
Этап ранжирования
выбранных кандидатов

Сравнительная таблица

	Кандидатогенерация	Ранжирование
Сколько айтемов может обрабатывать?	10к+ (размер каталога)	До 10к (реалистичнее до 1-3к)
Сколько признаков (факторов) может учитывать	Традиционно мало	Много
Время работы минимальное (в онлайн)	10-200 мс	10-200 мс
Время работы в оффлайне	+inf	+inf
Может ли использоваться самостоятельно (то есть формировать конечные рекомендации без другого этапа)?	Да, можно дальше не делать ранжирование	Да, например, если маленький каталог, или сортируем только самые популярные
Популярные представители	ALS, MF, SASRec, EASE, BERT4Rec	LGBM, Catboost, DCN
Как выглядят данные типичные?	Матрица взаимодействий, sequential данные	Табличные данные: базис + фичи + таргет
Ключевая метрика для оптимизации (обычно)	Recall@k	NDCG/MRR/MAP
Сложность реализации	Простая	Сложная

Проблемы матричной факторизации

Low-Rank Matrix Factorization:



Iterate:

$$f[i] = \arg \min_{w \in \mathbb{R}^d} \sum_{j \in \text{Nbrs}(i)} (r_{ij} - w^T f[j])^2 + \lambda ||w||_2^2$$

- 1) Не знают конверсии айтемов из просмотра в клик/покупку/просмотр до конца
- 2) Не знают, сколько раз юзер видел айтем этой категории
- 3) Не знают, как давно юзер смотрел этот айтем
- 4) Не знают, как часто юзеры этого возраста досматривали до конца айтемы из этой категории

...

Табличные данные – как основа для ранжирования



User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	1
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	1
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	0
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	0
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	1
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	1
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	0
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	0

Базис

Признаки

Таргет

Что такое базис?

Базис – это табличка вида:

- 1) **User_id** – или другой айди для которого надо выдать рекомендации: author_id, item_id (для i2i рекомендаций), business_id,
- 2) **Item_id** – это то, что мы ранжируем (category_id, partner_id, brand_id), и под item_id может подразумеваться music_id, video_id, sku_id и тд
- 3) **Datetime** (или просто date) – время, относительно которого мы разделяем будущее (какой таргет наступит) и прошлое (оттуда можем считать признаки)
- 4) **Target** – то, что модель должна предсказать в идеале точно.

Вопросы по поводу базиса

Базис – это табличка вида:

- 1) **User_id** – или другой айди для которого надо выдать рекомендации: author_id, item_id (для i2i рекомендаций), business_id,
- 2) **Item_id** – это то, что мы ранжируем (category_id, partner_id, brand_id), и под item_id может подразумеваться music_id, video_id, sku_id и тд
- 3) **Datetime** (или просто date) – время, относительно которого мы разделяем будущее (какой таргет наступит) и прошлое (оттуда можем считать признаки)
- 4) **Target** – то, что модель должна предсказать в идеале точно.

1) Что это за множество user_id/item_id, которые входят в базис?

2) Как собрать таргет?

Два основных пути

1) Что это за множество user_id/item_id, которые входят в базис?

User_id в момент времени datetime
увидел/услышал item_id на
поверхности в сервисе

User_id в момент времени datetime
имел item_id в кандидатах

- 1) «Видел» – это не подмножество «имел в кандидатах». Например, айтем мог быть не из рекомендаций просмотрен
- 2) Ранкер ранжирует кандидатов => в этом смысле логичнее вторая постановка
- 3) Однако негативные примеры более четкие те, которые юзер видел (первая постановка). Во втором случае можем считать негативами то, что юзер даже не видел (не знал)
- 4) Ранкер по первому варианту может не знать плохие примеры. Он ранжирует очень хорошие от просто хороших
- 5) Датасет во втором случае может быть больше, надо сэмплировать или будет тяжелее

Два основных пути



Какой путь лучше?

1) Что это за множество user_id/item_id, которые входят в базис?

User_id в момент времени datetime увидел/услышал item_id на поверхности в сервисе

User_id в момент времени datetime имел item_id в кандидатах

- 1) «Видел» – это не подмножество «имел в кандидатах». Например, айтем мог быть не из рекомендаций просмотрен
- 2) Ранкер ранжирует кандидатов => в этом смысле логичнее вторая постановка
- 3) Однако негативные примеры более четкие те, которые юзер видел (первая постановка). Во втором случае можем считать негативами то, что юзер даже не видел (не знал)
- 4) Ранкер по первому варианту может не знать плохие примеры. Он ранжирует очень хорошие от просто хороших
- 5) Датасет во втором случае может быть больше, надо сэмплировать или будет тяжелее

Сбор таргета

Как собирается таргет?

User_id в момент времени datetime увидел/услышал item_id на поверхности в сервисе, и сделал (target>0) /не сделал (target=0) с ним полезное действие

User_id в момент времени datetime имел item_id в кандидатах, и сделал (target>0) /не сделал (target=0) с ним полезное действие

- 1) То есть мы просто размечаем выбранный базис на полезное действие
- 2) Таргет может быть
 - 1) Бинарный – клик/не клик
 - 2) Композитный: 1 – кликнул, 2 – лайкнул, 3 – купил.
 - 3) «мультитаргет»: для каждого действия ставим 1/0: кликнул 1, лайкнул 1, купил 0

Задачи ранжирования и классификации

Если таргет бинарный, можем предсказать вероятность $y=1$ и потом отсортировать по убыванию вероятности
CTR task

Если таргет бинарный или небинарный, можем генерировать скор, который правильно отранжирует. То есть идеальный скор должен ранжировать по убыванию таргета
Ranking task

Если нам надо просто ранжировать айтемы, обычно лучше ranking task, даже на бинарном таргете.

CTR задача может быть полезна, если нам нужны откалиброванные вероятности действий. Например, чтобы оценить матожидание прибыли: цена * вероятность покупки товара. Часто это также используется в рекламе. Также CTR может быть полезен для триггерных рекомендаций, если скор становится больше трешхолда – отправляем коммуникацию с рекомендацией.

Но для ранжирования CTR обычно хуже!

В ранжировании мы получаем скоры от $-\infty$ до $+\infty$, но они задают порядок лучше, чем CTR

Выбор между задачами на уровне лосса, датасеты должны быть +- одинаковыми

Комбинированный таргет

Можем предсказывать для каждой пары user-item сразу (одной или разными моделями):

- 1) Вероятность клика p_1
- 2) Вероятность покупки p_2
- 3) Вероятность лайка p_3
- 4) Вероятность дизлайка p_4 .

Потом финальный скор релевантности составим как $w_1 * p_1 + w_2 * p_2 + w_3 * p_3 + w_4 * p_4$.

Как подобрать w_1-4 ?



Комбинированный таргет

Можем предсказывать для каждой пары user-item сразу (одной или разными моделями):

- 1) Вероятность клика p_1
- 2) Вероятность покупки p_2
- 3) Вероятность лайка p_3
- 4) Вероятность дизлайка p_4 .

Потом финальный скор релевантности составим как $w_1 * p_1 + w_2 * p_2 + w_3 * p_3 + w_4 * p_4$.

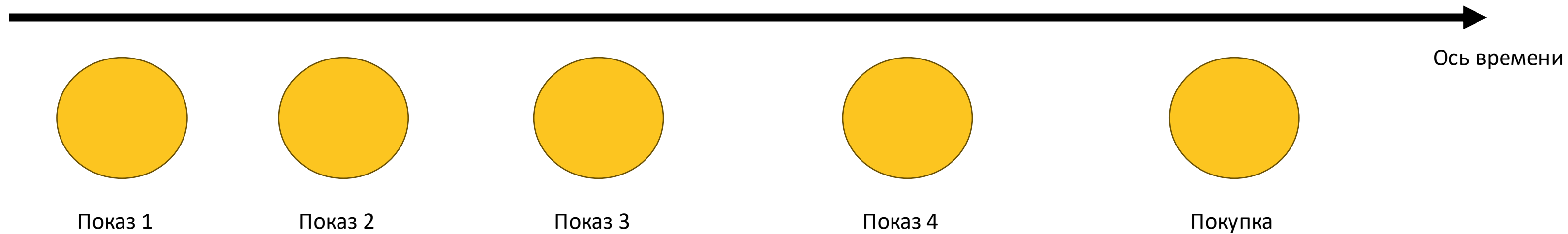
Как подобрать w_1-4 ?

- Экспертно
- Перебирая оптимальное значение метрики на валидации
- Можно подобрать экспериментально, в том числе через RL
- Можно придумать эвристики, типа доли в данных или из пересчета конверсии пропорционально

Этот подход позволяет осознанно растить разные метрики:

Выкручиваете вес у кликов, растите клики, остальное проседает. Либо растите покупки, но проседает все остальное

Сбор отложенного таргета



Сверху примеры истории по конкретному user-item

Произошла покупка, значит показ(ы) были не негативными.

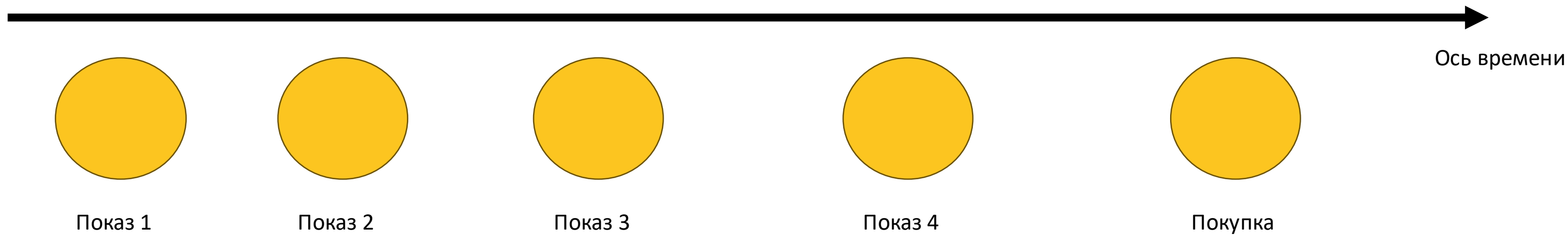
Существует понятие атрибуции – куда и как мы атрибуцируем покупку товара?

Варианты атрибуции:

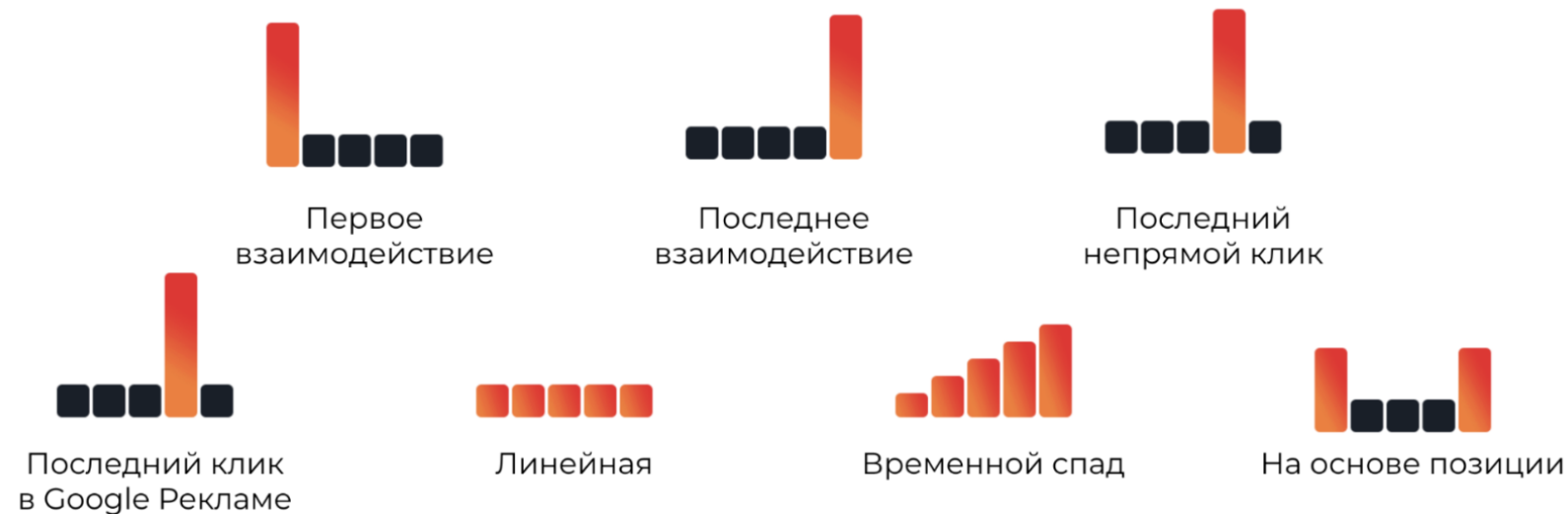
- 1) Первый показ
- 2) Последний показ
- 3) Всем показам поровну
- 4) Какой еще?



Про модели атрибуции



Модели атрибуции в Google Analytics:



channel_name	last_click	markov_chain	shapley_value
google_cpc	215	-0.1	0.2
GDN	99	-0.4	-0.5
yandex_cpc	95	-0.3	-0.2
email	65	-0.4	-0.1
direct	58	0.4	-0.3
affiliate	26	1.8	1.0
referral	11	3.5	2.2

Delta difference to base (last_click) model.

Advanced подход:

- 1) Строим модель атрибуции по показам
- 2) Модель распределяет вес каждого касания на целевое действие
- 3) Ранкер получает этот вес в виде target

Убеждаемся в адекватности базиса

User_id в момент времени datetime увидел/услышал item_id на поверхности в сервисе, и сделал (target>0) /не сделал (target=0) с ним полезное действие

User_id в момент времени datetime имел item_id в кандидатах, и сделал (target>0) /не сделал (target=0) с ним полезное действие

Идеальная модель идеально решает нашу задачу?



Надо предсказать юзеров, которые провзаимодействуют с автором в последнюю неделю.

Кандидаты – юзеры, которые взаимодействовали с автором в прошлом.

Есть история просмотров user-author.

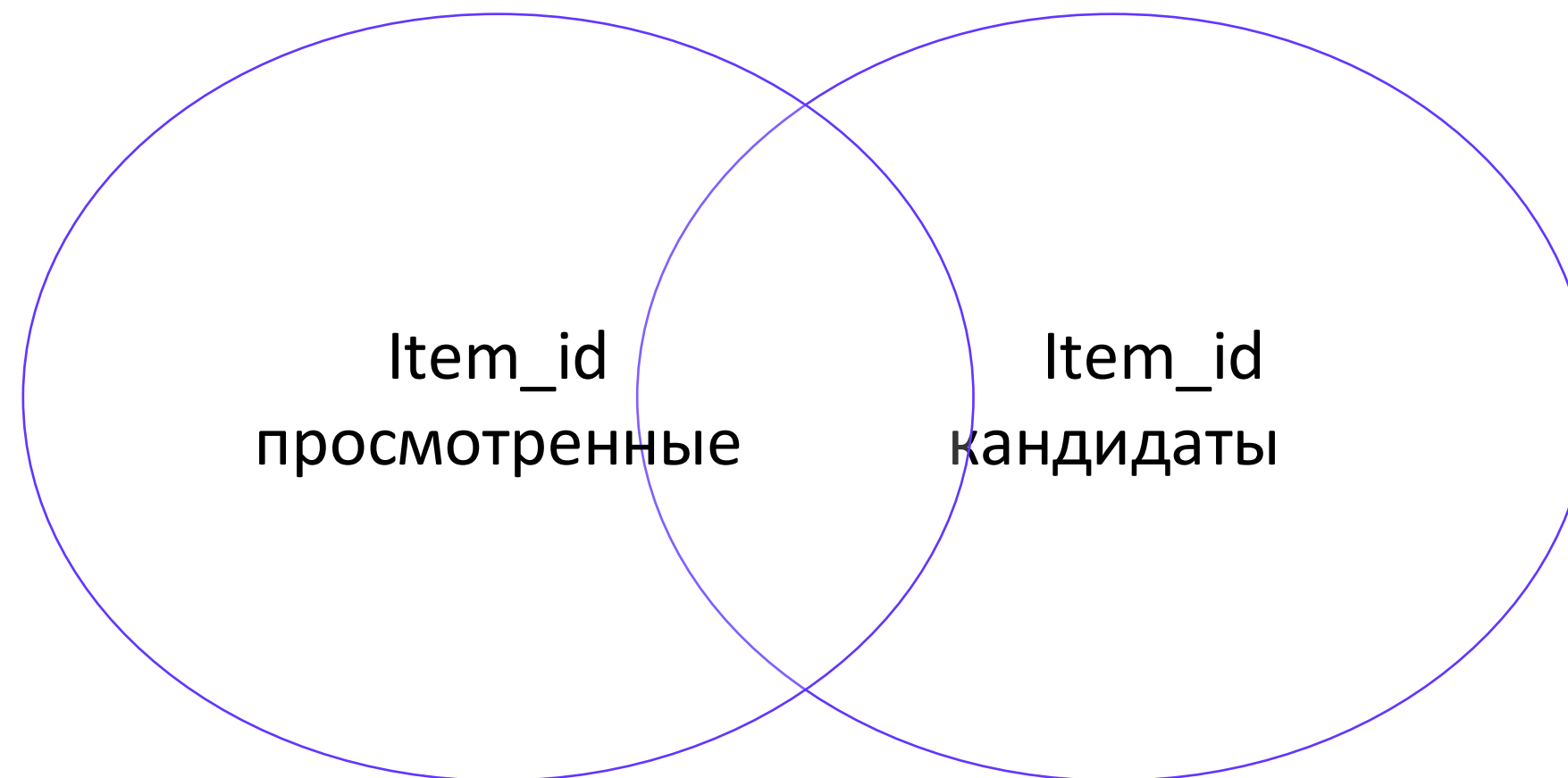


Лучше брать первый или второй вариант? (То есть берем историю показов за период времени и ранжируем эти показы, или размечаем на таргет кандидатов)?

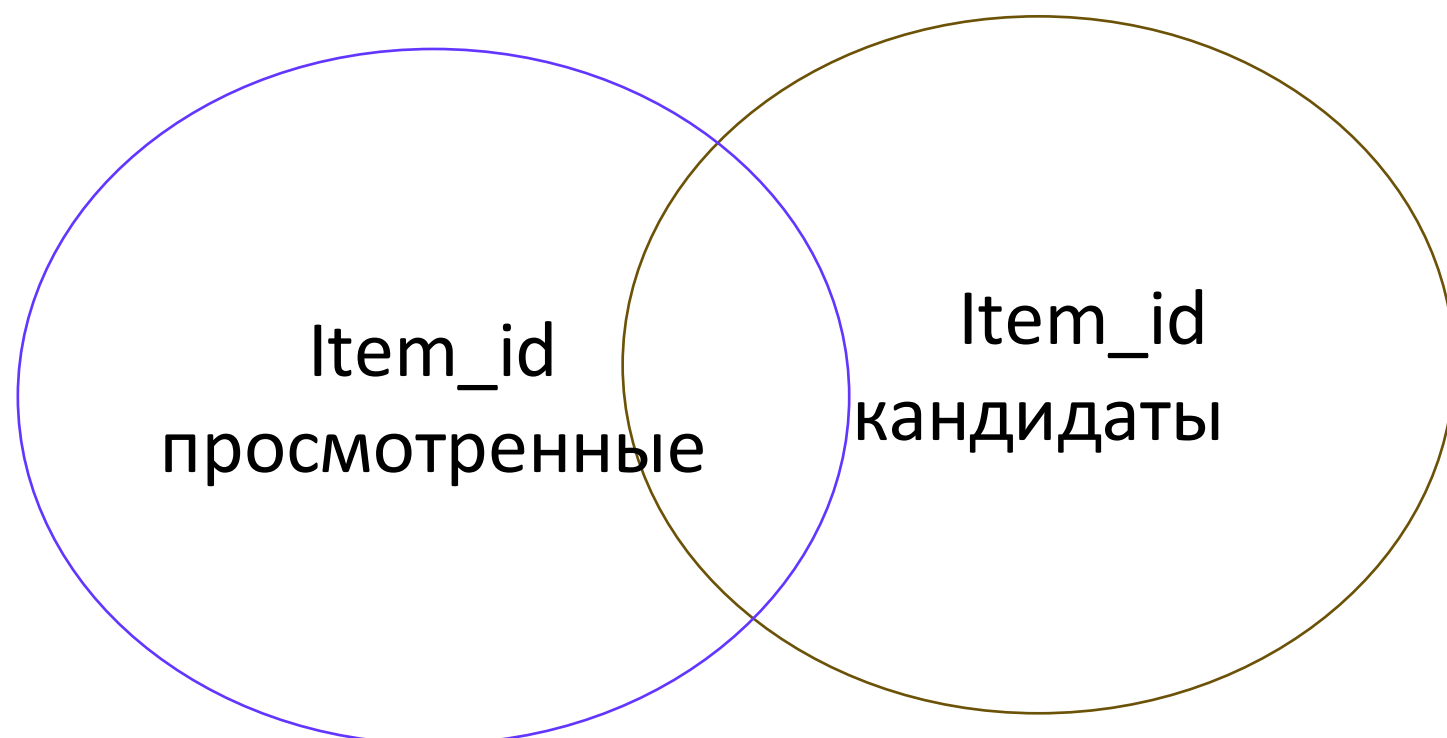
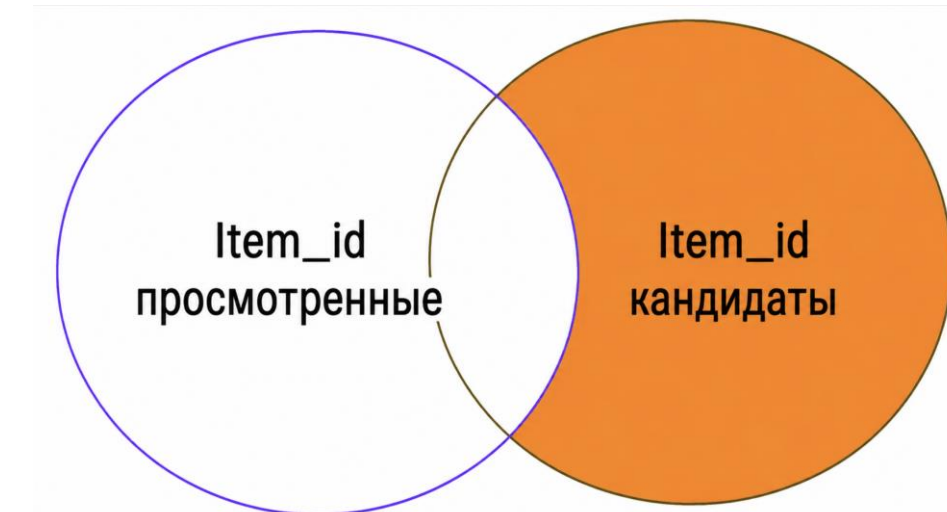
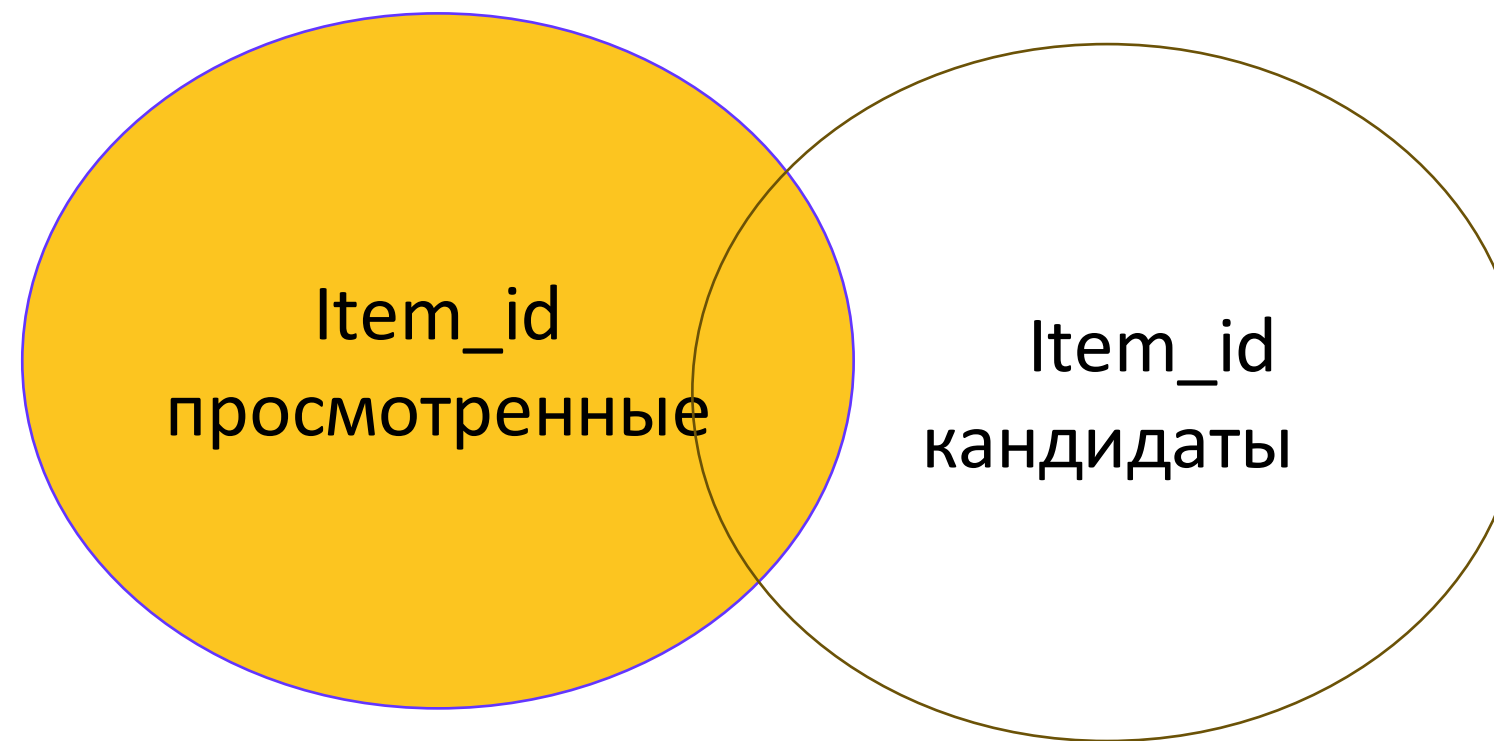
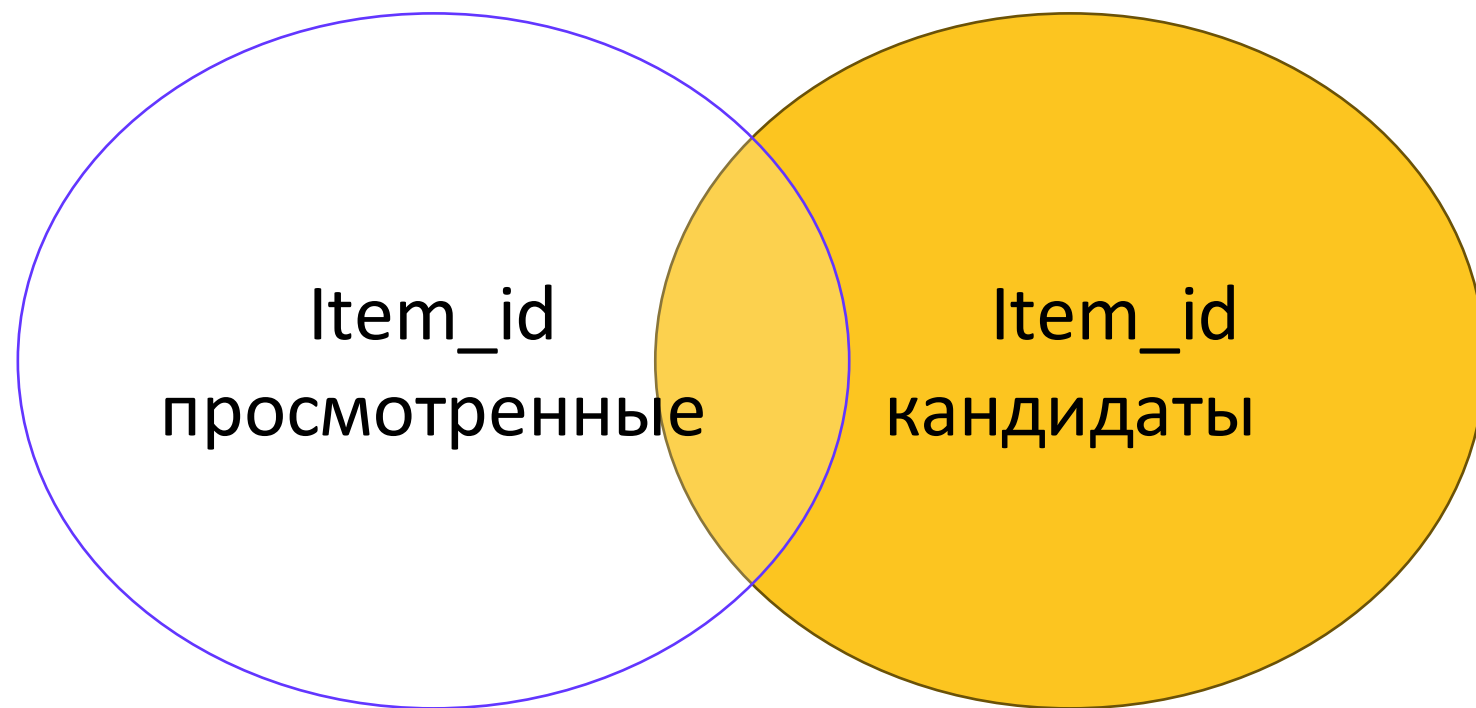
Еще про два вида сбора базиса

User_id в момент времени datetime увидел/услышал item_id на поверхности в сервисе, и сделал (target>0) /не сделал (target=0) с ним полезное действие

User_id в момент времени datetime имел item_id в кандидатах, и сделал (target>0) /не сделал (target=0) с ним полезное действие



Еще про два вида сбора базиса



Просмотренные где?

User_id в момент времени datetime увидел/услышал item_id на поверхности в сервисе, и сделал (target>0) /не сделал (target=0) с ним полезное действие

Посмотрел в ленте/поверхности рекомендаций

Плюсы:

- данные чисто с рекомендаций, лучше улавливаются паттерны взаимодействия с рекомендациями

Минусы:

- данных может быть меньше и недостаточно
- сильная зависимость от Feedback Loop

Посмотрел с любого места в сервисе (берем все просмотры)

Плюсы:

- много данных
- меньше зависимости от FL рекомендаций

Минусы:

- можно «зачерпнуть» паттернов поиска и т. Д.

Кейс: просмотренные из поиска?

User id в момент времени datetime увидел/услышал item id на поверхности в сервисе, и сделал (target>0) /не сделал (target=0) с ним полезное действие

Берем показы только из рекомендаций

Плюсы:

- Паттерны из рекомендаций

Минусы:

- Сильный feedback loop



Почему модели получатся разными?
(фичи одинаковы)

Берем только все показы из поиска

Плюсы:

- мы будем стараться показывать те айтемы, которые юзер будет искать в поиске до поиска

Минусы:

- каннибализация поиска
- может быть плохое качество

Кейс: просмотренные из поиска?

User id в момент времени datetime увидел/услышал item id на поверхности в сервисе, и сделал (target>0) /не сделал (target=0) с ним полезное действие

Берем показы только из рекомендаций

Плюсы:

- Паттерны из рекомендаций

Минусы:

- Сильный feedback loop



Почему модели получатся разными?
(фичи одинаковы)

Потому что из фичей они извлекут разную информацию

Берем только все показы из поиска

Плюсы:

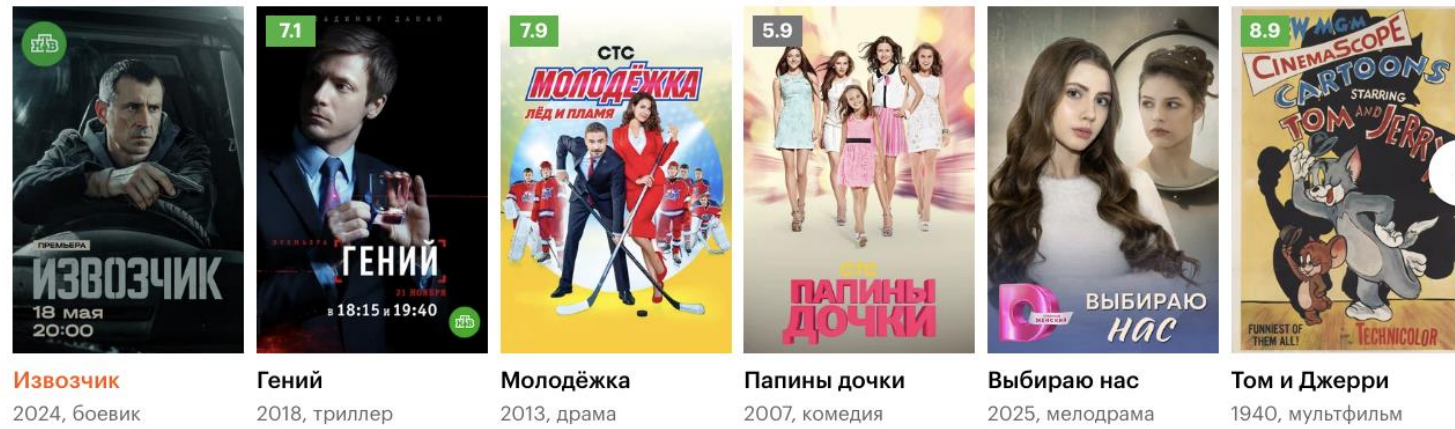
- мы будем стараться показывать те айтемы, которые юзер будет искать в поиске до поиска

Минусы:

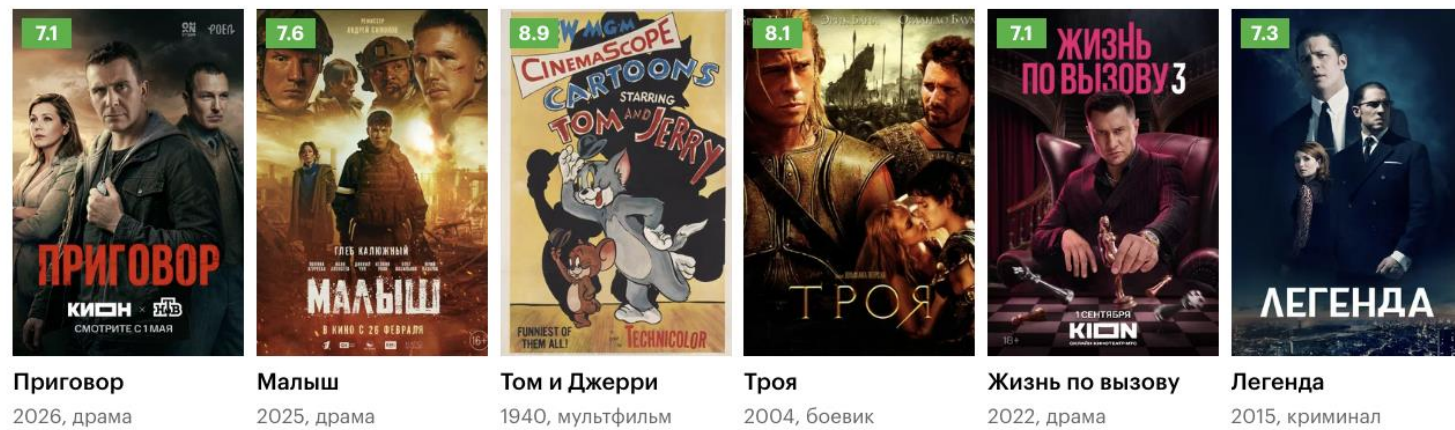
- каннибализация поиска
- может быть плохое качество

Кейс: просмотренные из конкретной ленты

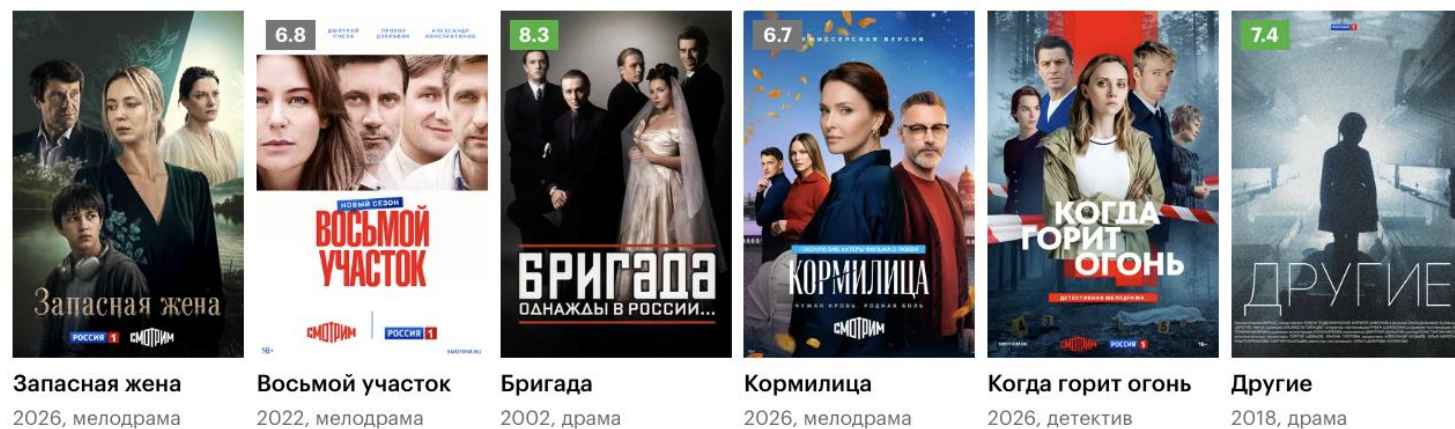
Смотрят на Иви



Смотрят на Кион



Смотрят на Смотрим



Можем:

1) Собрать одну модель на все подборки, и потом делать разных кандидатов.

2) Собрать модели под каждую подборку (базисы фильтруются на показы из нужной подборки).

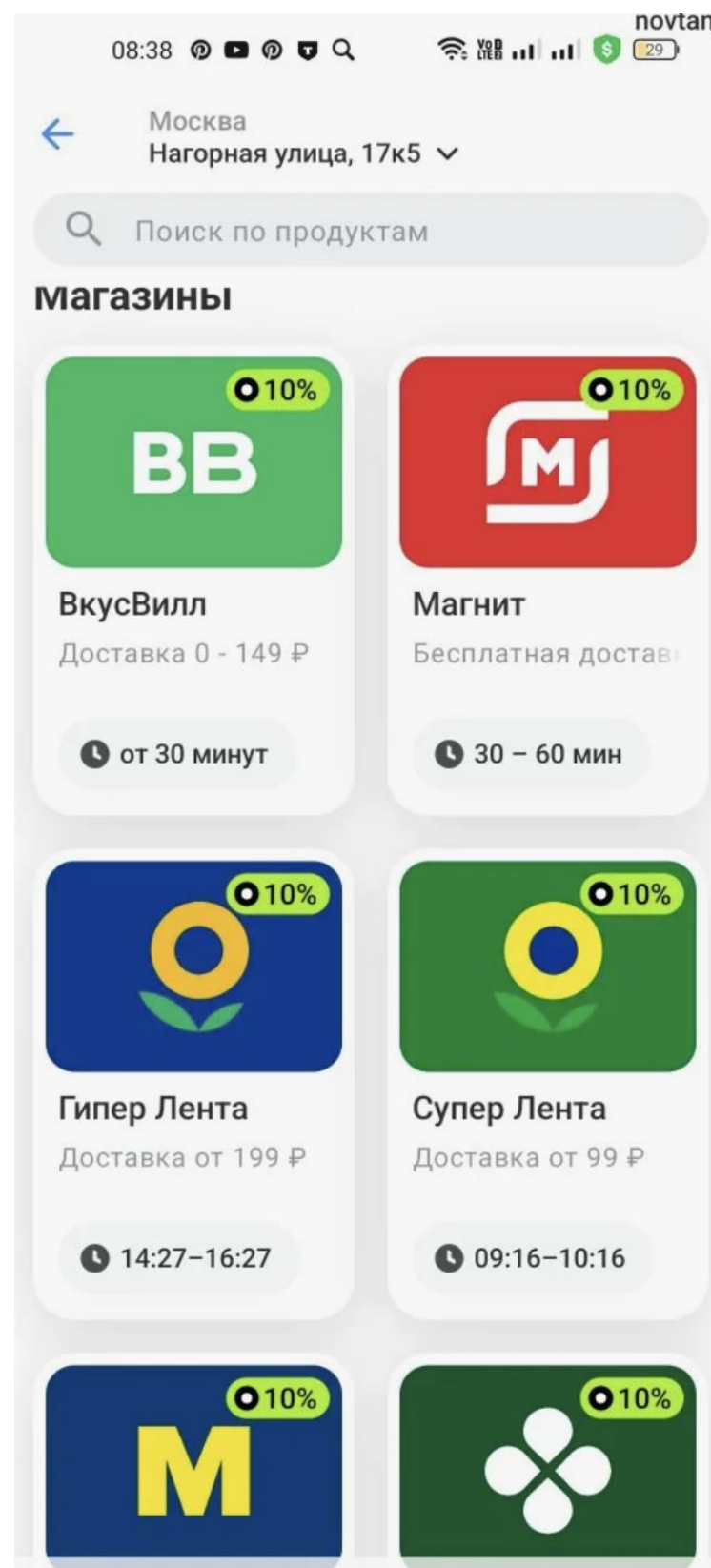
Плюсы по сравнению с п. 1:

- больше подстраиваемся под паттерны внутри подборки

Минусы:

- меньше данных, может быть хуже общей модели

Кейс: просмотренные из конкретного магазина



Можем:

- 1) Собрать одну модель на все магазины, и потом делать разных кандидатов.
- 2) Собрать модели под каждый магазин (базисы фильтруются на показы из нужной подборки).

В нашем случае лучше работал вариант 1, потому что данных внутри магазина было достаточно. Между магазинами разные паттерны, разные фишки важны и тд.

Группировка в базисе

User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	1
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	1
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	0
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	0
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	1
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	0
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	1
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	0

Pairwise ranking учится ранжировать объекты внутри группы!

Группировка в базисе

User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	1
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	1
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	0

Группировку можно сделать:

1) По date

1) Делаем (учим) ранжирование айтемов для юзера в течение выделенного дня/месяца/недели

2) По session_id (окно в 30 минут, например)

1) Делаем (учим) ранжирование айтемов для юзера в течение конкретной сессии

3) По запросу (выдаче рекомендательной, триггер показа рекомендаций)

1) Делаем (учим) ранжирование айтемов для юзера для показа конкретной ленты один раз

Группировка в базисе: риски

User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	1
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	1
User_id	Item_id	Item Features	User Features	User-item scores/rank	Context	0

Если у вас сэмплы собраны за разное время внутри одной группы – это риск дата лика по фичам.

Если один сэмпл пользователя за вчера, второй за сегодня, то в фичах на сегодня может быть информация о юзере за вчера, что явно подскажет модели, хороший пример «вчера» или нет

Базис – (user_id, item_id, date, target) – это основа.
Плохой базис, хорошие фичи – плохая модель
Хороший базис, Плохие фичи – нормальная модель
Хороший базис, хорошие фичи – хорошая модель

Основные типы признаков (фичей)

1) Item –фичи посчитаны только по конкретному item	Item Features	User Features	User-item scores/rank	Context
2) User фичи посчитаны по конкретному user	Item Features	User Features	User-item scores/rank	Context
3) User-item фичи – посчитаны для конкретной пары user-item. Так называемые кросс-фичи. Одни из самых важных	Item Features	User Features	User-item scores/rank	Context
4) Context - фичи контекста (время, девайс, локация, погода)	Item Features	User Features	User-item scores/rank	Context
 Какие еще фичи можно придумать?	Item Features	User Features	User-item scores/rank	Context

Основные типы признаков (фичей)

1) Item –фичи посчитаны только по конкретному item

Item Features

User Features

User-item scores/rank

Context

2) User фичи посчитаны по конкретному user

Item Features

User Features

User-item scores/rank

Context

3) User-item фичи – посчитаны для конкретной пары user-item. Так называемые кросс-фичи. Одни из самых важных

Item Features

User Features

User-item scores/rank

Context

4) Context - фичи контекста (время, девайс, локация, погода)

Item Features

User Features

User-item scores/rank

Context

5) User-context/item-context/user-item/context – кросс фичи по трем типам

Item Features

User Features

User-item scores/rank

Context

Item Features

User Features

User-item scores/rank

Context

Общие свойства всех признаков

Надо думать от общего к частному. Каждый признак вы можете:

- 1) Внутри продукта или вовне продукта. Популярность фильма в нашем сервисе и популярность фильма на Кинопоиске – разная степень внешности
- 2) Можете посчитать за разные периоды времени – за 1 час/1 день/7 дней/год/5 лет, обычно относительно текущего времени. Это разные признаки, но с одним смыслом. Большое окно = длинный контекст и хороший шанс что-то вспомнить нужное. Маленькое окно – вы всегда используете самые актуальные данные и вовремя забываете старые
- 3) Можете посчитать с разной поверхности – со всех сразу/только с рекомендаций/поиска/каталога/конкретных подборок/карточках товара и т. Д. Это значит, что вы берете данные только из конкретных источников, остальное игнорируете
- 4) Вы можете считать как абсолютные счетчики, так и отношения между какими-то признаками (например, число кликов / число показов)
- 5) Можно считать «число дней с последнего... до текущего момента», «число дней с первого... до текущего»

Общие свойства всех признаков

Признаки могут быть:

- 1) Вещественными – хорошо ложится в бустинги и нормально в нейросети
- 2) Категориальными – чуть хуже ложится в бустинги и хорошо в нейросети
- 3) Просто эмбедингами – почти не стоит использовать в бустингах и хорошо можно использовать в нейросетях

Общее число признаков:

Плюс минус нормальное значение 200-1000 признаков, больше число уже вряд ли принесет хороший профит

Признаки **должны быть** посчитаны по данным до datetime/date из нужного сэмпла в базисе! Иначе будет даталик и ничего не будет хорошо работать

Item признаки

Вещественные:

- число просмотров (показов)/кликов/лайков/покупок/дослушиваний/досматриваний за последние N дней в сервисе X с поверхности Y
(далее эту длинную вариативность для каждой фичи упоминать не буду)
- конверсии из просмотра в клик/добавление в корзину, конверсии из добавления в корзину в выкуп и т. Д.
- средняя цена или скидка, время просмотра, доля дослушиваний
- Время доставки, доля возвратов, рейтинг, число отзывов

Категориальные:

- сам item_id
- category_id/partner_id/author...

Эмбединги:

- текстовый/cv эмбединг от других моделей

Item Features

Item Features

Item Features

Item Features

Item Features

Item Features

Item Features

Item Features

Item признаки обобщенные

Item_id - конкретный item. Но обычно есть еще более абстрактная категоризация, явно привязанная к item:

- author_id (поста)
- partner_id (селлер)
- merchant_id (партнер)
- category_id (соответствующая категоризация).

Далее по каждому этому более абстрактному “item_id” считаем такие же фичи, как к item_id

Вещественные:

- число просмотров (показов)/кликов/лайков/покупок/дослушиваний/досматриваний за последние N дней в сервисе X с поверхности Y
(далее эту длинную вариативность для каждой фичи упоминать не буду)
- конверсии из просмотра в клик/добавление в корзину, конверсии из добавления в корзину в выкуп и т. Д.
- средняя цена или скидка, время просмотра, доля дослушиваний
- Время доставки, доля возвратов, рейтинг, число отзывов

Категориальные:

- сам item_id
- category_id/partner_id/author...

Эмбединги:

- текстовый/cv эмбединг от других моделей

Item Features

Item Features

Item Features

Item Features

Item Features

Item Features

Item Features

Item Features

Зачем item фичи для ранжирования?

В основном модели используют item признаки для оценки популярности/надежности/общей статистической вероятности, что один айтем лучше другого на основе прошлых результатов

Item Features

Модели любят популярные айтемы, потому что популярность часто переносится из прошлого в будущее. Люди покупали бананы весь месяц чаще всего – скорее всего и в обучающей выборке банан был выбран вместо киви (этот вывод можно сделать чисто по item фичам популярности)

Item Features

Item Features

Item Features

Item фичи не отвечают за персонализацию

Item Features

Item Features

Item Features

Item Features

User фичи

User-id – конкретный юзер, под которого мы ранжируем

User Features

Вещественные:

- число просмотров (показов)/кликов/лайков/покупок/дослушиваний/досматриваний

User Features

- конверсии из просмотра в клик/добавление в корзину, конверсии из добавления в корзину в выкуп и т. Д.

User Features

- средняя цена или скидка, время просмотра, доля дослушиваний

User Features

- Время доставки, доля возвратов, рейтинг, число отзывов

User Features

Категориальные:

- сам user_id

User Features

- gender/age/city/education/...

User Features

User Features

User фичи обобщенные

User-id – конкретный юзер, под которого мы ранжируем

Аналогично, users принадлежат к некоторым кластерам, и по кластерам можно считать фичи.

Кластера:

- gender
- age
- city/country
- income....

User Features

User Features

User Features

User Features

User Features

User Features

User Features

User Features

Зачем user фичи в ранжировании?

На самом деле user фичи могут плохо помочь для ранжирования.

User Features

Если мы решаем задачу классификации (предсказания вероятности), тогда user фичи помогают отсеять высокоактивных клиентов (с высокими значениями фичей) от низкоактивных, и лучше откалибровать вероятность на основе юзера.

User Features

User Features

Если мы решаем задачу ранжирования для юзера, то все фичи внутри группы одинаковы. По ним сложно дифференцировать релевантность, потому что внутри группы фичи не отличаются. В целом это возможно, но в редких кейсах и на практике для ранжирования дает мало профита.

User Features

User Features

Если у вас есть два айтема (банан и молоко) и вы говорите, что человек мужчина. Сложно на основе этой фичи сказать, что банан лучше молока

User Features

User Features

User Features

User-item фичи (самые важные)

Вещественные:

- число просмотров (показов)/кликов/лайков/покупок/дослушиваний/досматриваний
- конверсии из просмотра в клик/добавление в корзину, конверсии из добавления в корзину в выкуп и т. Д.
- средняя цена или скидка, время просмотра, доля дослушиваний
- Время доставки, доля возвратов, рейтинг, число отзывов
- **Скоры от моделей кандгенов: ALS, BERT4Rec, TIFU KNN... -> супер важно**

Если в данных есть повторные взаимодействия user-item, то если хоть где-то есть не 0, модель этот айтем будет воспринимать лучше, чем айтемы, с которыми не было взаимодействия.

Какая вероятность, что модель угадает один айтем из 10к, с которыми не было взаимодействия у юзера и 1 айтем из 100, с которыми юзер взаимодействовал в прошлом и может взаимодействовать еще?

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item фичи (самые важные)

- Скоры от моделей кандгенов: ALS, BERT4Rec, TIFU KNN... -> супер важно

User-item scores/rank

Есть повторы в user-item или нет, не играет роли. Модели ALS/BERT4Rec/TIFU KNN/... помогают оценить релевантность на основе всех действий юзера обобщенно!

User-item scores/rank

Эти модели могут быть единственными ненулевыми персональными фичами по user-item, поэтому вся персонализация может лечь на них:

User-item scores/rank

Представьте, что user-category/user-author/user-partner – везде нулевые значения фичей (юзер ничего не делал с категорией/автором/партнером айтема до этого), но user_ALS_item скор будет ненулевой, так как этот скор оценит, насколько история юзера похожа на этот айтем конкретно.

User-item scores/rank

User-item scores/rank

User-item scores/rank

Эти скоры должны быть посчитаны без даталика, то есть в скорях не должно зашито быть информации из того же времени >datetime (это время сбора базиса), иначе будет даталик

User-item scores/rank

User-item scores/rank

User-item фичи обобщенные

Все то же самое + можем считать als/bert4rec/... по user-category (они должны работать по матрице взаимодействий/действий на уровне user-category).

Тут могут быть “ключи”:

- User-partner
- User-author
- User-category
- User-city...

Зачем это нужно? Если user-item фичи нулевые – в домене нельзя делать повторные рекомендации айтемов юзерам – или нас они не интересуют – тогда фича user-category поможет извлечь из истории пользователя что-то похожее на интерес к айтемам.

Например, мы знаем, что user-категория фрукты – имеет высокое значение, значит возможно ему будут релевантны бананы (хотя он их никогда не покупал)

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

User cluster –item cluster фичи обобщенные

Все то же самое, но мы обобщаем людей до сегментов людей:

Тут могут быть “ключи”:

- gender-partner
- gender-author
- gender-category
- gender-city...
- Gender-age-category
- Gender-city-income-category/.....

Например, даем модели понять, что косметику покупают в основном женщины, и если заходит женщина, рекомендуем косметику повыше, а если мужчина – пониже. Эта информация извлекается автоматически с помощью подобных фичей.

Если вы добавите просто фичи типа gender и category как категориальные – модели намного сложнее уловить эту связь с «женщинами и косметикой», чем если вы явно заложите ее в фичи gender-category

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

User-item scores/rank

Context

Context – фичи про актуальность в момент времени предсказания, но что может меняться и должно быть важно учесть:

- время суток/года/день месяца
- девайс человека
- экран (плейсмент)
- локация человека

Эти фичи можно вставлять категориально и надеяться, что модель сама разберется, но это не очень удачный вариант (удачный на следующем слайде).

Контекст обычно нужно учитывать через онлайн рекомендации, которые считаются в момент захода юзера и учитывают актуальный контекст. Если вы работаете в оффлайне (предсчитанные рекомендации), здесь можно насчитать рекомендации для каждого значения контекста и в зависимости от него показывать нужные рекомендации, но это сложная работа

Context

Context

Context

Context

Context

Context

Context

Context

User-Context/Item-context

User-item-context, примеры:

- Сколько раз юзер покупал этот товар утром?
- Сколько раз юзер слушал эту песню в машине?
- Сколько раз юзер смотрел этого автора в воскресенье?
- Сколько раз юзер покупал этот товар, когда было жарко?

Представьте ситуацию, захотите в жаркую погоду в сервис доставки продуктов, а там мороженое – этого можно добиться через такие фишки!

Item-context, примеры:

- популярность айтема в это время суток/в этой локации/в воскресенье (этот день недели)
- конверсия в покупку в этот [context].

Есть домены, где разные контексты важны (время в музыке, день недели в видео рекомендациях, время дня в продуктах питания). А где-то неважны.

Такие фишки дорого считать и собирать, но они могут принести классный эффект персонализации

Context

Context

Context

Context

Context

Context

Context

Context

User history

Item 1/action 1

Item 2/action 2

Item 3/action 3

Item 4/action 4

Вы можете рассмотреть историю юзера как набор его предыдущих действий (последовательность).

Последовательность кодируется через трансформер в эмбединг (например по последнему действию).

Эмбединг вы трактуете как эмбединг пользователя (user feature) и дальше все как обычно.

Плюсы:

- модель может более гибко связывать действия пользователя с конкретным айтемом сейчас для рекомендаций

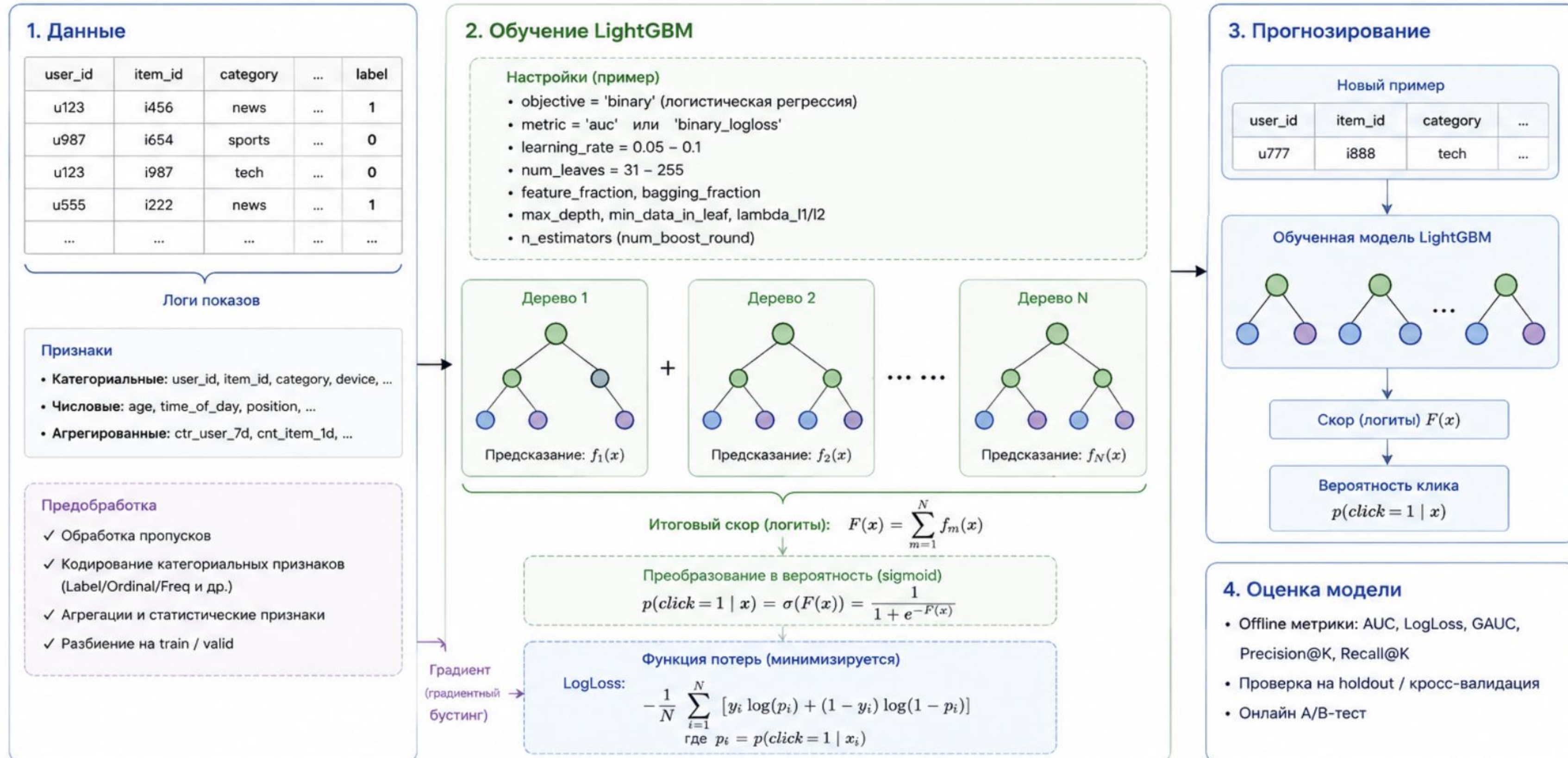
Минусы:

- это усложняет систему заметно как для обучения, так и для инференса

Не нейросетевое ранжирование: LGBM

Обучение CTR-модели через LightGBM

Цель: предсказать $p(\text{click}=1 \mid \text{user}, \text{item}, \text{context})$



Обучение через LambdaRank в LightGBM (Learning to Rank)

Цель: научить модель ранжировать документы для запроса, оптимизируя NDCG@k

1. Данные (парные/групповые)

Каждая группа = один запрос (query) и набор документов (items)

query_id	doc_id	relevance (y)	features ...
q1	d1	3	...
q1	d2	2	...
q1	d3	0	...
q1	d4	1	...
...	...	...	...
q2	d5	2	...
q2	d6	0	...
q2	d7	3	...
...	...	...	...

Группа (query) q1

Группа (query) q2

2. Текущие предсказания модели

На входе в итерацию t модель имеет скор: $s_i^{(t)}$

Пример для запроса q1

doc_id	y (rel)	скор $s_i^{(t)}$
d1	3	1.20
d2	2	0.30
d3	0	-0.40
d4	1	0.10

Текущий ранжированный список (по $s_i^{(t)}$)

1	d1 (rel=3)	1.20
2	d2 (rel=2)	0.30
3	d4 (rel=1)	0.10
4	d3 (rel=0)	-0.40

Целевой идеальный порядок (по rel)

1	d1 (rel=3)	
2	d2 (rel=2)	
3	d4 (rel=1)	
4	d3 (rel=0)	

d1 релев.=3 (наиболее релевантный) d2 релев.=2 d4 релев.=1 d3 релев.=0 (наименее релевантный)

3. Lambda-градиенты (λ): знаки и формулы

Рассматриваем упорядоченную пару документов (i, j) с разной релевантностью.

Для пары с $y_i > y_j$:

$$\rho_{ij} = \frac{1}{1 + e^{\sigma(s_i - s_j)}}$$
$$\lambda_{ij}^{(i)} = -\sigma |\Delta NDCG_{ij}| \cdot \rho_{ij}$$
$$\lambda_{ij}^{(j)} = +\sigma |\Delta NDCG_{ij}| \cdot \rho_{ij}$$

Знак зависит от того, какой объект в паре рассматриваем: более релевантный объект получает отрицательный вклад (его score нужно повысить), менее релевантный — положительный вклад (его score нужно понизить).

4. Агрегация λ в градиенты по документам

Каждый документ получает суммарный лямбда-градиент:

$$\lambda_i = \sum_{j \neq i} \lambda_{ij}$$

Пример для q1 (t)

doc_id	λ_i (градиент)
d1	-0.85
d2	-0.25
d4	+0.30
d3	+0.80

отрицательный $\lambda_i \rightarrow$ поднять выше / увеличить score

положительный $\lambda_i \rightarrow$ опустить ниже / уменьшить score

Поэтому итоговая λ_i может быть как отрицательной, так и положительной.

5. Использование λ в Gradient Boosting (LightGBM)

LightGBM на каждой итерации t строит дерево по псевдоградиентам, полученным из лямбда-значений.

$$\mathcal{L}^{(t)} = \sum_{i=1}^N \lambda_i f_t(x_i) + \frac{1}{2} \sum_{i=1}^N |g_i| f_t(x_i)^2 + \Omega(f_t)$$

псевдоградиент (λ_i) вес (hessian proxy): $|g_i|$ регуляризация дерева

Здесь $g_i = \lambda_i$ (псевдоградиент), $h_i = |\lambda_i|$ (вес).

LightGBM интуитивно: дерево учится предсказывать поправку к score так, чтобы уменьшать ranking loss и улучшать NDCG.

Обновление скоров (после построения дерева f_t):

$$s_i^{(t+1)} = s_i^{(t)} + \eta f_t(x_i) \quad (0 < \eta \leq 1 - \text{learning rate})$$

6. Обучение дерева на λ -градиентах

LightGBM строит регрессионное дерево, аппроксимируя псевдоградиенты (цели):

$$target_i = \lambda_i = - \sum_{j \neq i} \sigma |\Delta NDCG_{ij}| \rho_{ij}^{(i)}$$

с весами $h_i = |\lambda_i|$.

Дерево разбивает пространство признаков так, чтобы в листах значения $f_t(x)$ наилучшим образом аппроксимировали псевдоградиенты.

$$Gain = \frac{1}{2} \left(\frac{(\sum_{i \in \mathcal{L}_{left}} \lambda_i)^2}{\sum_{i \in \mathcal{L}_{left}} |\lambda_i| + \lambda_2} + \frac{(\sum_{i \in \mathcal{L}_{right}} \lambda_i)^2}{\sum_{i \in \mathcal{L}_{right}} |\lambda_i| + \lambda_2} - \gamma \right)$$

7. Обновление весов (скоров)

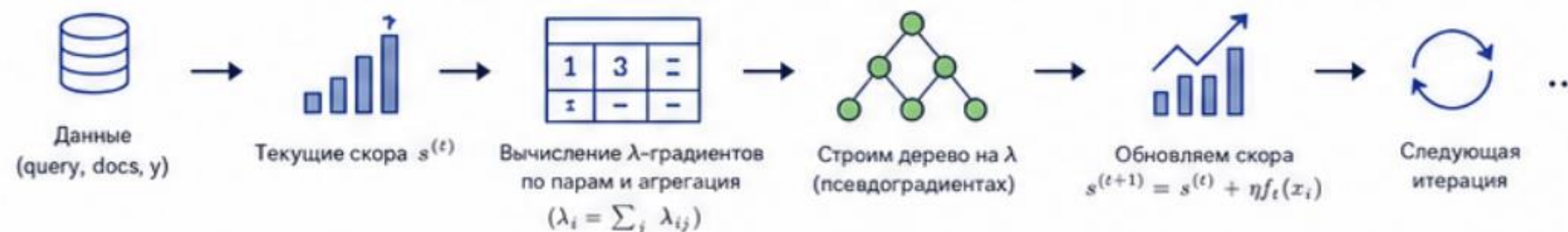
После построения дерева $f_t(x)$ обновляем score для каждого документа:

$$s_i^{(t+1)} = s_i^{(t)} + \eta f_t(x_i)$$

где η — learning_rate.

Затем переходим к следующей итерации (t+1) и повторяем шаги 2–7.

Итеративный процесс LambdaRank в LightGBM



Что оптимизируется?



LambdaRank в LightGBM оптимизирует релевантные метрики ранжирования (чаще всего NDCG@k) через градиентный бустинг на λ -градиентах.

DeepFM для CTR Prediction

Совместное обучение low-order и high-order взаимодействий признаков

1. Входные данные и признаки

Поле	Пример значения
user_id	100235
item_id	50219
category	books
device	mobile
hour	15
age	25
price_bin	3
label	1 (click)

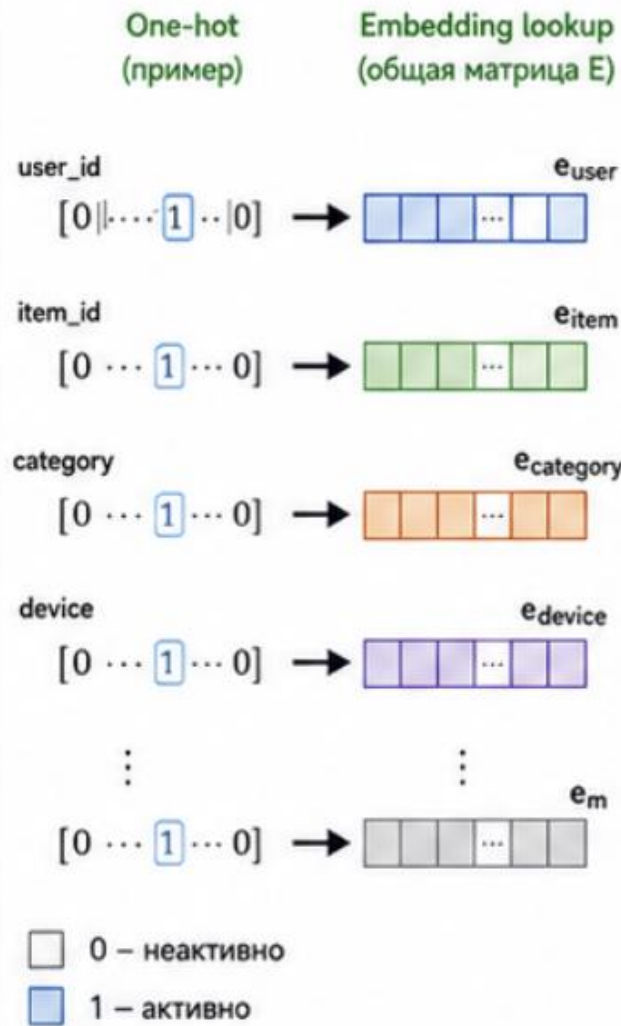
- Категориальные признаки: user_id, item_id, category, device, ...
- Числовые признаки: price, age, hour, ...
- Обычно все признаки представляются как sparse / one-hot поля
- Цель: предсказать $p(\text{click} = 1 | \text{user, item, context})$



Что такое x ?

x — входной вектор признаков одного объекта (user, item, context) после кодирования.
 x_i — значение i -го признака: для one-hot/categorical обычно 0 или 1, для числовых признаков — число (часто нормализованное). В CTR задаче x соответствует объекту (user, item, context) после кодирования признаков.

2. Общий слой embedding



Shared input: одна и та же embedding-представление используется в FM и Deep частях.

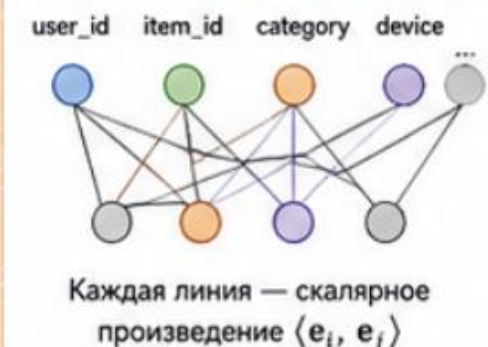
3. FM-компонента (low-order interactions)

FM моделирует 1-го и 2-го порядка взаимодействия признаков.

$$y_{\text{FM}} = w_0 + \sum_i w_i x_i + \sum_{i < j} \langle e_i, e_j \rangle x_i x_j$$

- Линейные эффекты признаков
- Парные взаимодействия через скалярные произведения embedding-векторов
- Хорошо ловит memorization / sparse co-occurrence

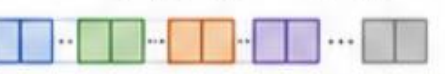
Парные взаимодействия (пример)



4. Deep-компонента (high-order interactions)

Конкатенация всех embedding-векторов и прохождение через MLP.

$$a^{(0)} = [e_1; e_2; \dots; e_m]$$



Dense (128) + ReLU

Dense (64) + ReLU

Dense (32) + ReLU

$a^{(L)}$

$$a^{(l+1)} = \text{ReLU}(W^{(l)} a^{(l)} + b^{(l)})$$

$$y_{\text{Deep}} = W_{\text{out}} a^{(L)} + b_{\text{out}}$$

- Изучает сложные нелинейные взаимодействия
- Хорошо ловит generalization
- Не требует ручного feature engineering crosses

5. Выход модели

Складываем выходы FM и Deep.

$$y = y_{\text{FM}} + y_{\text{Deep}}$$

$$p(\text{click} = 1 | x) = \sigma(y)$$

$$\sigma(y) = \frac{1}{1 + e^{-y}}$$

Функция потерь:

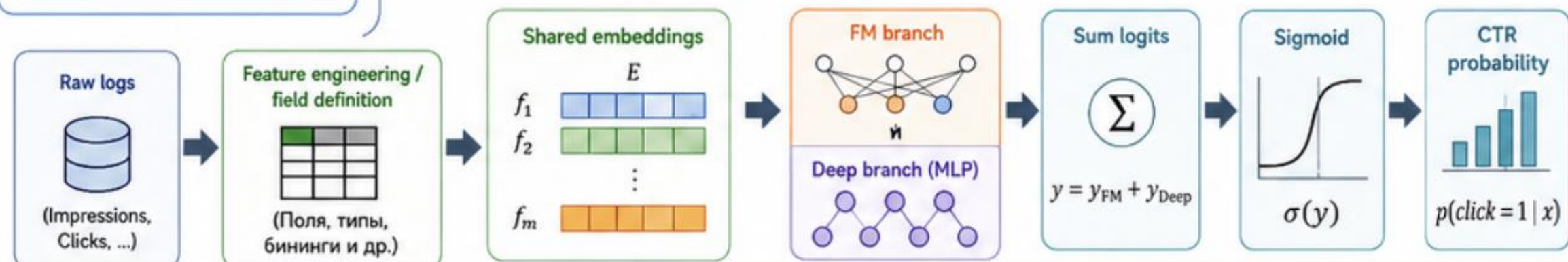
LogLoss / Binary Cross-Entropy

$$L = -\frac{1}{N} \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Оценка модели:

- AUC
- LogLoss
- GAUC
- Calibration / online A/B

Поток работы DeepFM



Ключевая идея DeepFM

- ✓ FM отвечает за low-order feature interactions
- ✓ Deep часть — за high-order interactions
- ✓ Оба блока обучаются end-to-end без отдельного pretraining

Wide & Deep для CTR Prediction

Совместное обучение memorization и generalization для предсказания клика

1. Входные данные и признаки

Поле	Пример значения
user_id	u123
item_id	i456
category	books
device	mobile
hour	19
age	25
price_bin	3
label	1 (click)

- Категориальные признаки: user_id, item_id, category, device, ...
- Числовые признаки: age, price, hour, ...
- Контекстные признаки: time, platform, position, ...
- Цель: предсказать $p(\text{click} = 1 \mid \text{user, item, context})$



Что такое x ?

x — входной вектор признаков объекта (user, item, context) после кодирования;
 x_i — отдельный признак.

2. Wide-компонента (memorization)

Линейная модель по исходным и cross-признакам.

$$y_{\text{wide}} = w^T [x, \phi(x)] + b$$

где $\phi(x)$ — cross-признаки, созданные вручную (feature engineering).

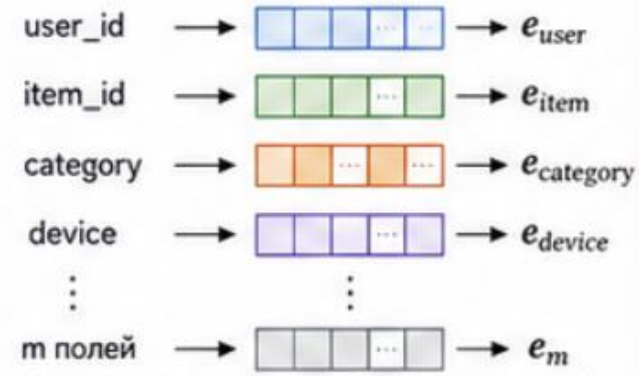
- Линейная модель по исходным и cross-признакам
- Хорошо запоминает частые паттерны и совместные появления
- Требуется feature engineering cross-ов

Примеры cross-features

- × (user_id = u123) × (item_id = i456)
- × (device = mobile) × (category = books)
- × (hour_bin = evening) × (category = video)

3. Deep-компонента (generalization)

Sparse-признаки кодируются в эмбединги и подаются в MLP.



$$a^{(0)} = [e_1; e_2; \dots; e_m]$$

Dense (128) + ReLU

Dense (64) + ReLU

Dense (32) + ReLU

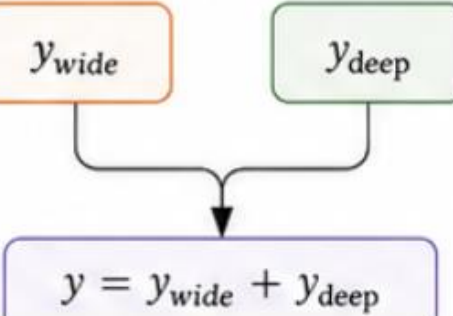
$$a^{(L)}$$

$$y_{\text{deep}} = W_{\text{out}} a^{(L)} + b_{\text{out}}$$

- Embedding-представления sparse-признаков
- Изучает сложные нелинейные взаимодействия
- Лучше обобщает на редкие и новые комбинации признаков

4. Объединение Wide и Deep

Суммируем логиты двух ветвей.



$$p(\text{click} = 1 \mid x) = \sigma(y)$$

$$\sigma(y) = \frac{1}{1 + e^{-y}}$$



Wide = memorization,
Deep = generalization

Обе части обучаются совместно end-to-end.

5. Обучение и оценка

Оптимизируем модель по логистической функции потерь.

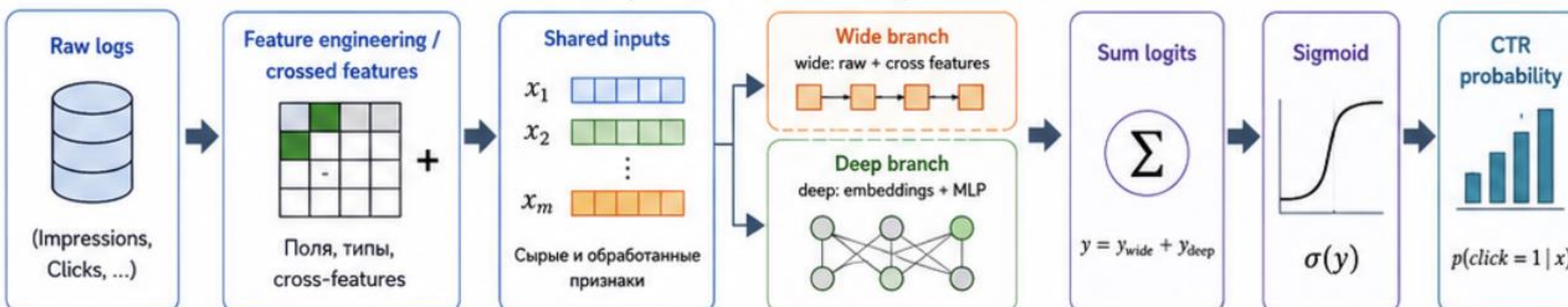
LogLoss / Binary Cross-Entropy

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log (1 - p_i)]$$

- Оптимизация: SGD / Adam и backpropagation
- Метрики: AUC, LogLoss, GAUC
- Проверка: offline holdout + online A/B
- Выход модели: вероятность клика pCTR



Поток работы Wide & Deep



Ключевая идея Wide & Deep

- ✓ Wide часть запоминает полезные частые cross-паттерны
- ✓ Deep часть обобщает на новые сочетания признаков
- ✓ Совместное обучение улучшает CTR prediction

YoutubeDNN

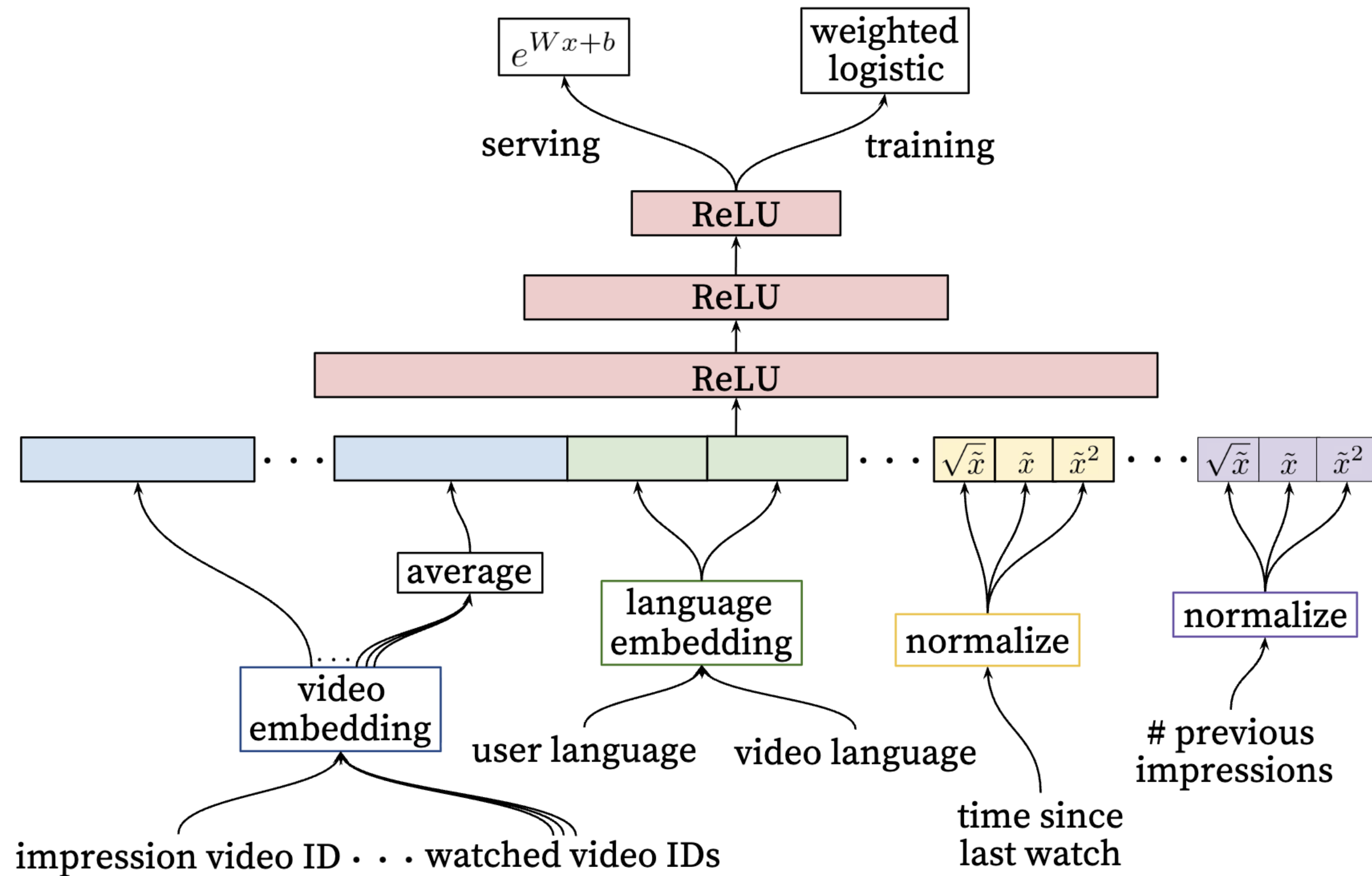


Figure 7: Deep ranking network architecture depicting embedded categorical features (both univalent and multivalent) with shared embeddings and powers of normalized continuous features. All layers are fully connected. In practice, hundreds of features are fed into the network.

DCN V2

DCN v2 Cross Layer — игрушечный пример

1. Входные признаки

$$x_0 = \begin{bmatrix} u \\ i \\ p \end{bmatrix}^T = \begin{bmatrix} 2 \\ 1 \\ 0.5 \end{bmatrix}^T$$



u — user



i — item

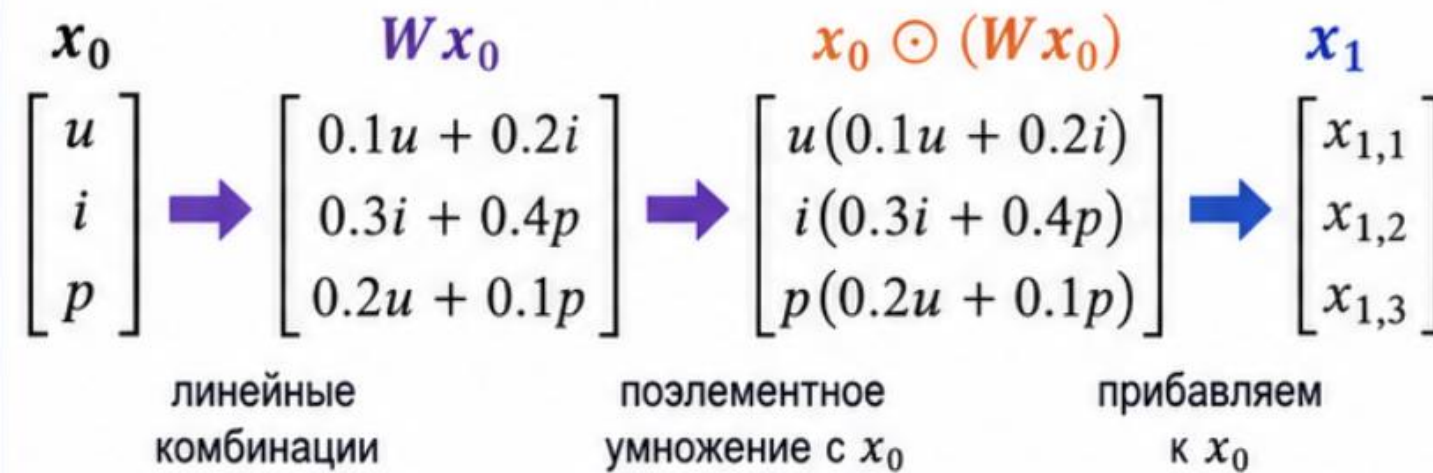


p — discount

2. Cross layer

$$x_1 = x_0 + x_0 \odot (Wx_0 + b)$$

для простоты $b = 0$



$$W = \begin{bmatrix} 0.1 & 0.2 & 0 \\ 0 & 0.3 & 0.4 \\ 0.2 & 0 & 0.1 \end{bmatrix}$$

3. Символьный результат

$$Wx_0 = \begin{bmatrix} 0.1u + 0.2i \\ 0.3i + 0.4p \\ 0.2u + 0.1p \end{bmatrix} \quad \Bigg| \quad x_1 = \begin{bmatrix} u + 0.1u^2 + 0.2ui \\ i + 0.3i^2 + 0.4ip \\ p + 0.2up + 0.1p^2 \end{bmatrix}$$



Cross layer явно строит **пересечения признаков**

4. Числовой пример

Вход: $x_0 = \begin{bmatrix} 2 \\ 1 \\ 0.5 \end{bmatrix}$

$$Wx_0 = \begin{bmatrix} 0.4 \\ 0.5 \\ 0.45 \end{bmatrix}$$

$$x_0 \odot (Wx_0) = \begin{bmatrix} 0.8 \\ 0.5 \\ 0.225 \end{bmatrix}$$

$$x_1 = \begin{bmatrix} 2.8 \\ 1.5 \\ 0.725 \end{bmatrix}$$



- 1 cross layer → взаимодействия **2-го порядка** (ui , ip , up)
- 2 cross layers → взаимодействия **3-го порядка**



DCN v2 сочетает
Cross Network и Deep Network

DCN V2 для CTR Prediction

Explicit feature crosses через Cross Network + deep generalization

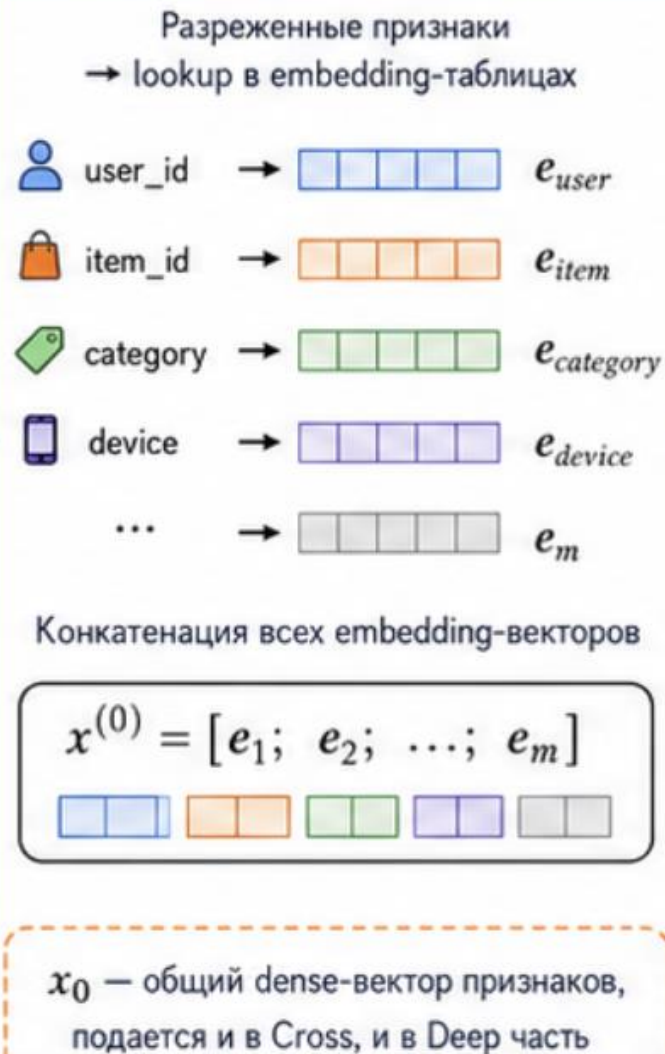
1. Входные данные и признаки

Поле	Пример значения
user_id	u123
item_id	i456
category	books
device	mobile
hour	19
age	25
price_bin	3
label	1 (click)

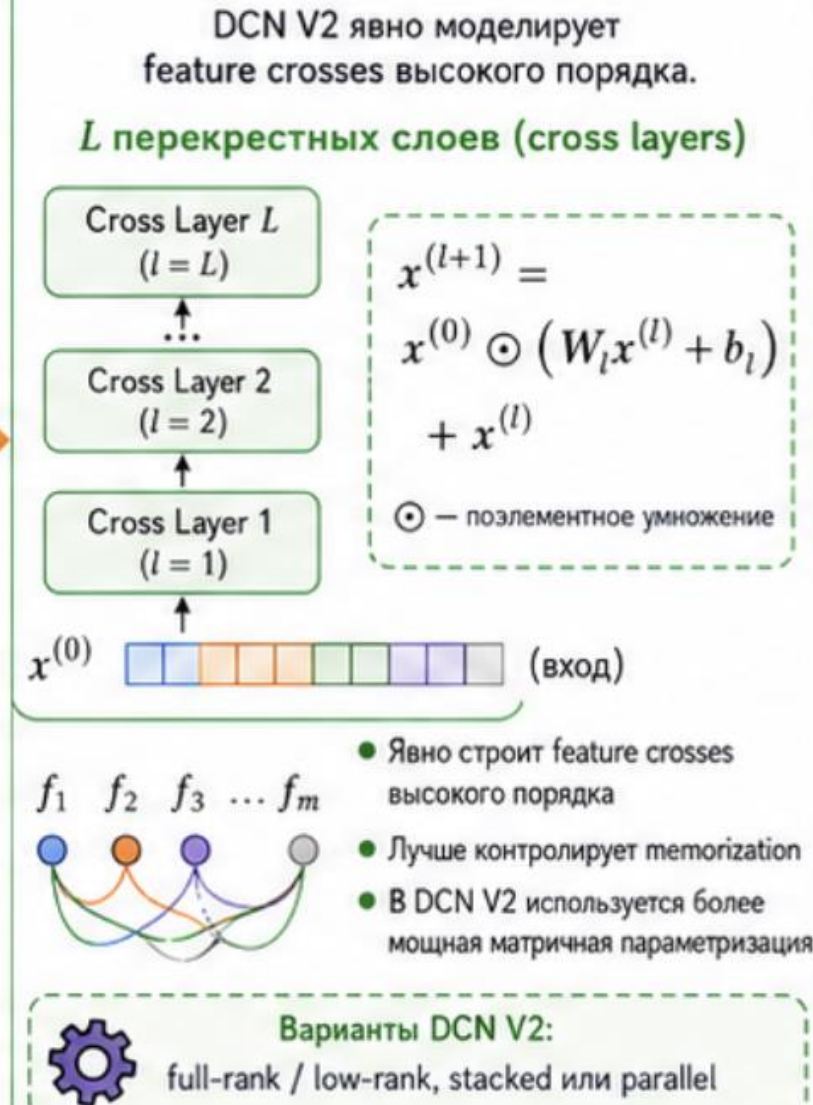
- Категориальные признаки: user_id, item_id, category, device, ...
- Числовые признаки: age, price, hour, ...
- Контекстные признаки: time, platform, position, ...
- Цель: предсказать $p(\text{click} = 1 | \text{user, item, context})$

Что такое x ?
 x — входной вектор признаков объекта после кодирования;
 x_i — отдельный признак.

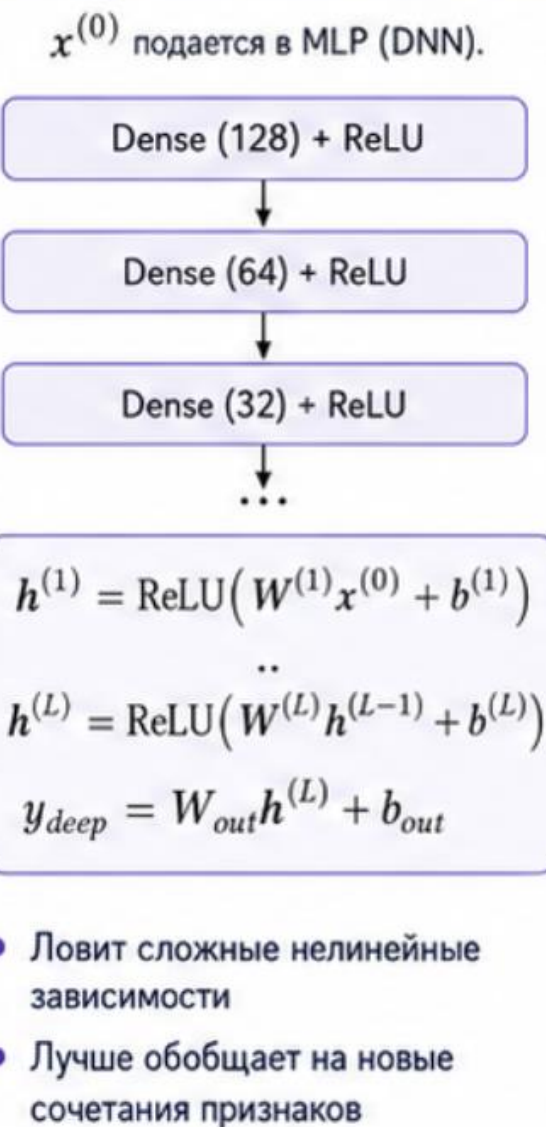
2. Embedding и базовый вектор x_0



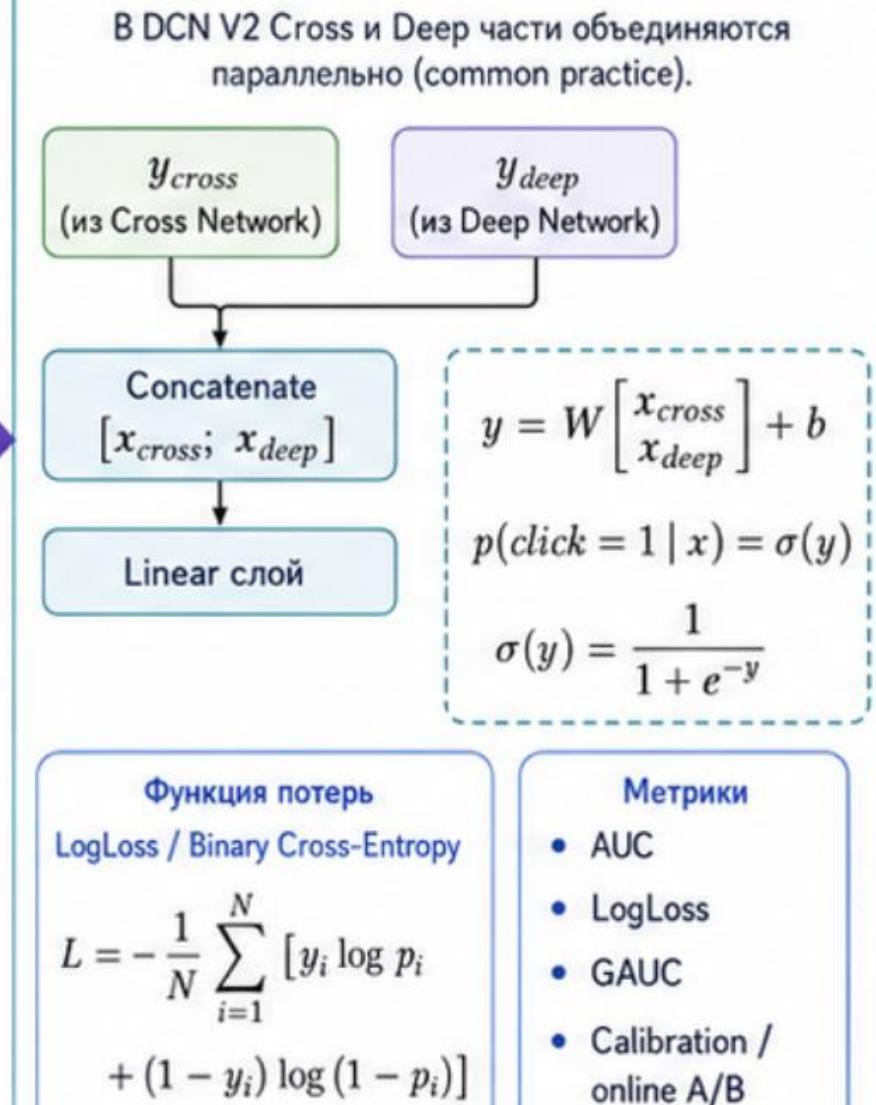
3. Cross Network (explicit feature interactions)



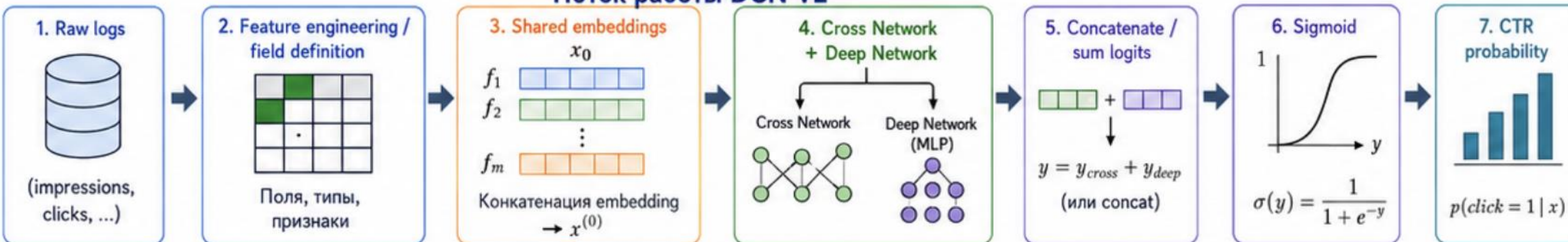
4. Deep-компонента (implicit interactions)



5. Объединение и выход модели



Поток работы DCN V2



Ключевая идея DCN V2

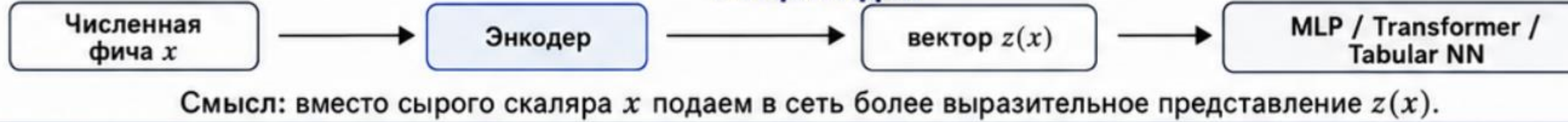
- ✓ Cross-сеть явно строит feature crosses
- ✓ Deep-часть отвечает за generalization
- ✓ Совместное end-to-end обучение улучшает CTR prediction

PLE / PLR: кодирование численных фи́ч в нейросетях

Как превратить скаляр x в информативный векторный эмбе́динг

! Важно: здесь PLR = Periodic + Linear + ReLU (по Gorishniy et al.), а не piecewise linear regression.

Общая идея



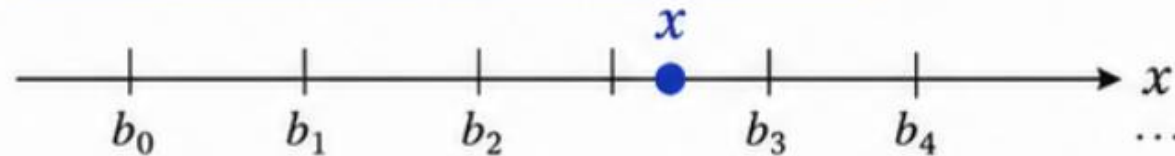
1) PLE (Piecewise Linear Encoding)

- 1 Задаем границы бинов: $b_0 < b_1 < \dots < b_T$ (всего T бинов)
- 2 Находим бин t , куда попало значение x
- 3 Если $x \in [b_t, b_{t+1})$, то

$$PLE(x) = [1, \dots, 1, \alpha, 0, \dots, 0]$$

$$\alpha = \frac{x - b_t}{b_{t+1} - b_t}$$

Границы бинов на оси x



Векторное представление (T компонент)



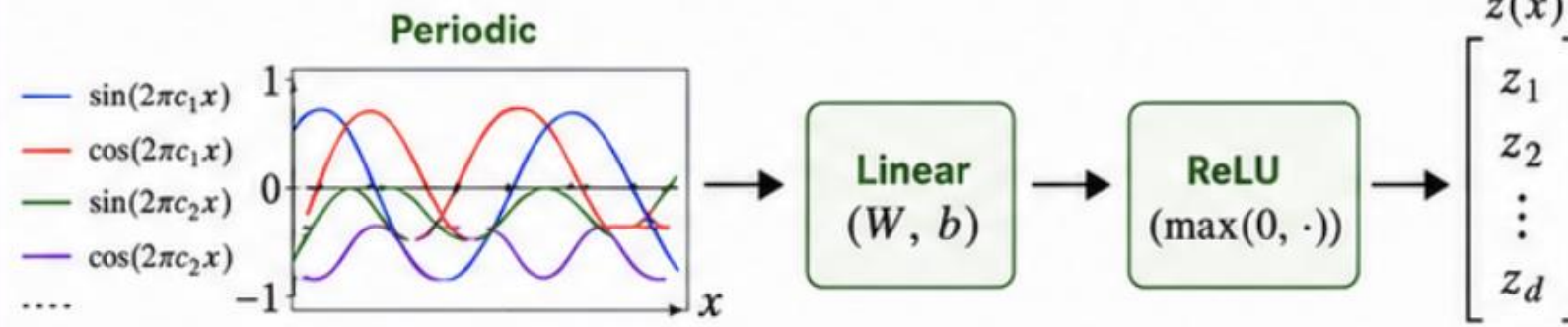
PLE сохраняет порядок и расстояние внутри текущего бина. Это термометр-подобное кусочно-линейное кодирование.

2) PLR (Periodic + Linear + ReLU)

- 1 **Periodic**: переводим x в набор периодических признаков
- 2 **Linear**: смешиваем их линейным слоем
- 3 **ReLU**: добавляем нелинейность

$$\text{Periodic}(x) = [\sin(2\pi c_1 x), \cos(2\pi c_1 x), \dots, \sin(2\pi c_k x), \cos(2\pi c_k x)]$$

$$PLR(x) = \text{ReLU}(W \cdot \text{Periodic}(x) + b)$$



Периодические функции помогают выразить сложные, гладкие и повторяющиеся зависимости по x .

3) Когда что использовать?

Метод	Как устроен	Сильные стороны	Когда полезен
Raw x	Сырой скаляр	Просто и дешево	Если зависимость почти линейная
PLE	Бины + кусочно-линейное кодирование	Хорошо учитывает локальные пороги и порядок	Табличные данные, пороговые/локальные эффекты
PLR	Periodic → Linear → ReLU	Гибкое гладкое представление	Когда нужны более сложные нелинейные зависимости



Оба подхода превращают одно число x в более богатое представление $z(x)$, которое затем используется вместе с эмбе́ддингами остальных признаков.

Multi Hashing Trick для эмбедингов категориальных фичей

Как кодировать большие категориальные признаки с меньшим числом параметров

1. Проблема

Категориальный id

x

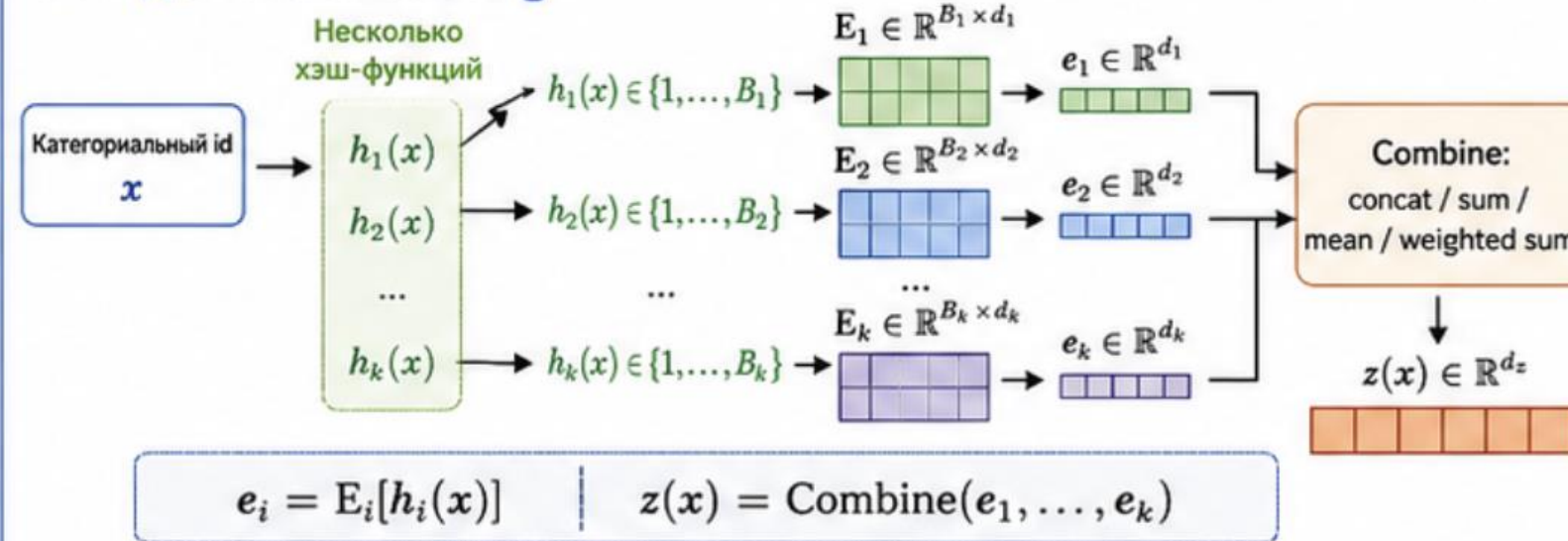
cat_1
cat_2
cat_3
...
cat_M
...
cat(|V|)

Огромная embedding-таблица
размера $|V| \times d$

id	1	2	...	d
1	$e_{1,1}$	$e_{1,2}$	...	$e_{1,d}$
2	$e_{2,1}$	$e_{2,2}$	...	$e_{2,d}$
...	...	...	...	...
V -1	$e_{ V -1,1}$	$e_{ V -1,2}$	...	$e_{ V -1,d}$
V	$e_{ V ,1}$	$e_{ V ,2}$	...	$e_{ V ,d}$

- Большой словарь категорий: $|V|$ может быть очень большим
- Обычный embedding требует таблицу размера $|V| \times d$
- Это дорого по памяти

2. Идея multi hashing



- Вместо одной огромной таблицы используем несколько маленьких
- Разные хэши уменьшают вред от коллизий

3. Как это работает

- 1 Берём категориальный id x
- 2 Пропускаем через несколько независимых хэш-функций
- 3 Ищем векторы в нескольких embedding-таблицах
- 4 Объединяем их в итоговый эмбединг

Хэш-функции

$$h_1(x) = 2$$

$$h_2(x) = 7$$

$$h_3(x) = 4$$

Embedding-таблицы

$E_1 \in \mathbb{R}^{B_1 \times d_1} \quad (B_1 = 8, d_1 = 4)$						
idx	1	2	3	4	...	8
1	0.1	0.3	0.7	-0.2	...	0.4
2	0.5	-0.1	0.2	0.9	...	-0.3
3	-0.2	0.0	0.6	-0.1	...	0.8
...	...	...	...	...	...	...
8	0.3	-0.4	0.1	0.2	...	0.6

$E_2 \in \mathbb{R}^{B_2 \times d_2} \quad (B_2 = 10, d_2 = 4)$									
idx	1	2	3	4	5	6	7	8	9
...	...	...	...	...	...	...	...	...	...
7	7	0.6	-0.3	0	0.1	...	...	...	...
...	...	...	...	...	...	...	...	...	...

$E_3 \in \mathbb{R}^{B_3 \times d_3} \quad (B_3 = 8, d_3 = 4)$							
idx	1	2	3	4	5	6	7
...	...	...	...	...	...	...	...
4	-0.6	0.2	0.2	0.4	-0.1	...	...

$$e_1 \in \mathbb{R}^4$$

$$e_2 \in \mathbb{R}^4$$

$$e_3 \in \mathbb{R}^4$$

Combine
(concat / sum / mean / weighted sum)

$$z(x) \in \mathbb{R}^{d_z}$$

Сравнение числа параметров

- Обычный embedding: $|V| \times d$ параметров
- Multi hashing: $\sum_i (B_i \times d_i)$ параметров
- Обычно $\sum_i B_i \ll |V|$

4. Плюсы и минусы

Плюсы

- Сильно экономит память
- Подходит для очень больших словарей
- Можно использовать несколько маленьких таблиц вместо одной огромной
- Снижает влияние коллизий по сравнению с одним hashing trick

Минусы

- Коллизии всё равно возможны
- Нужно выбирать число хэшей k и размеры таблиц B_i
- При слишком маленьких таблицах качество может падать
- Concat увеличивает размер выхода

5. Когда использовать

Метод	Память	Коллизии	Комментарий
Обычный embedding	высокая	нет	лучше качество, если память не ограничена
Один hashing trick	низкая	выше	самый простой вариант
Multi hashing	низкая/средняя	ниже	хороший компромисс



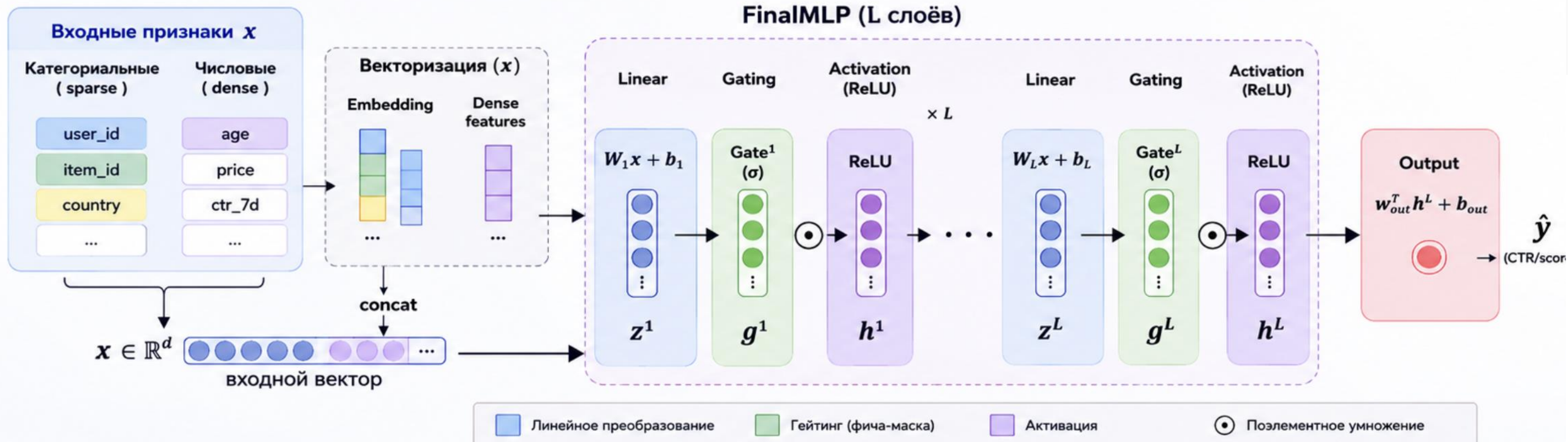
Полезно для CTR, рекомендательных систем и табличных моделей с большими категориальными признаками.



Идея: один id кодируется несколькими хэшами и несколькими embedding-таблицами, а затем их векторы объединяются в одно компактное представление.

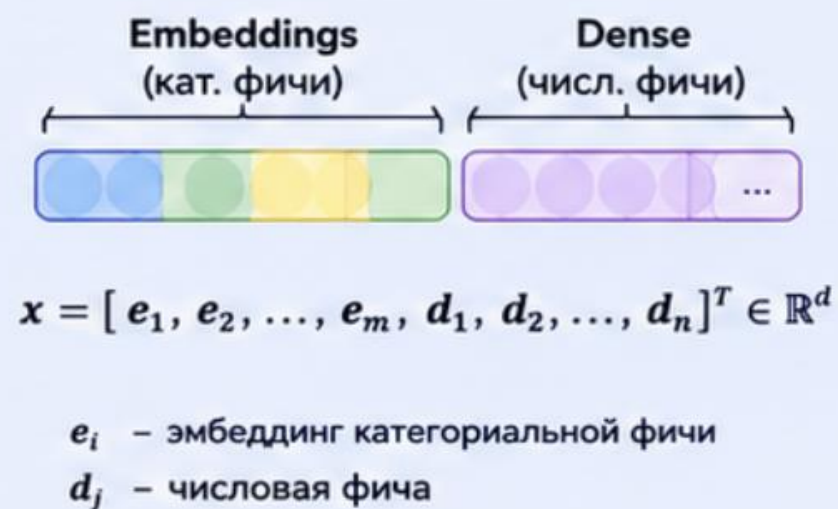
FinalMLP

MLP с поэлементным gating-механизмом, который решает, какие фичи пропускать дальше



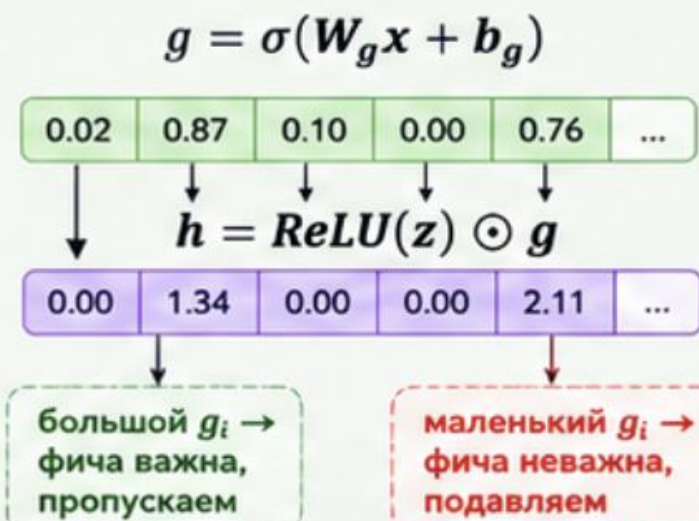
Что такое x ?

Единый входной вектор модели.



Что такое gating?

Гейтинг решает, какие признаки должны пройти дальше.

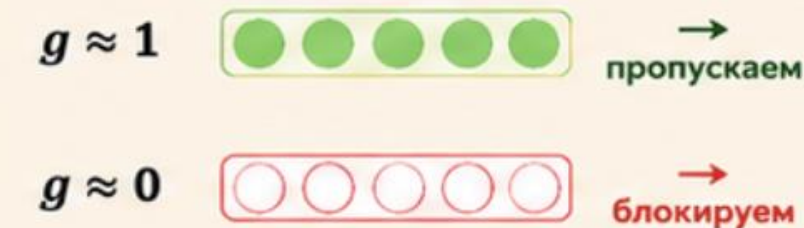


Как работает один слой?

1. Линейное преобразование
 $z = W x + b$
2. Гейтинг (σ – сигмоида в $[0,1]$)
 $g = \sigma(W_g x + b_g)$
3. Поэлементное умножение (маска)
 $z \odot g$
4. Активация (ReLU)
 $h = \text{ReLU}(z \odot g)$

Интуиция

Gating действует как «клапан» для каждой скрытой единицы.



Преимущество: модель сама учится, какие признаки нужны в каждом слое.

DIN для CTR Prediction

Deep Interest Network: target-aware attention по истории пользователя

1. Входные данные и признаки

Поле	Пример значения
user_id	u123
target_item	i456
category	books
device	mobile
hour	19
label (click)	1

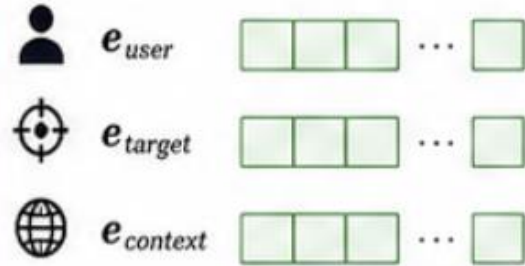
- categorical features (user, item, category, device...)
- numerical / context features (hour, dayofweek, position...)
- user behavior history (просмотры / клики)

Что такое x ?

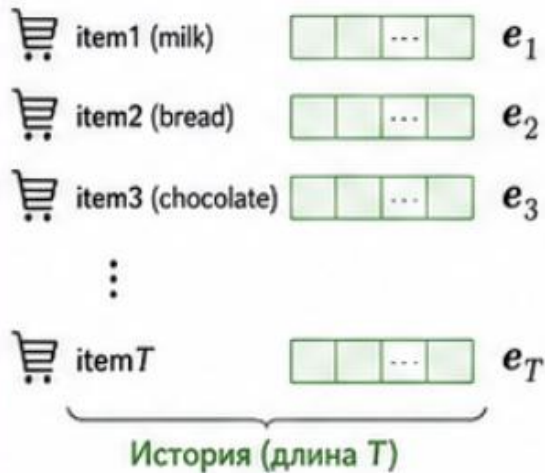
x — набор признаков текущего показа: user, target item, context, history.

2. Эмбединги

Разреженные поля отображаются в эмбединги.



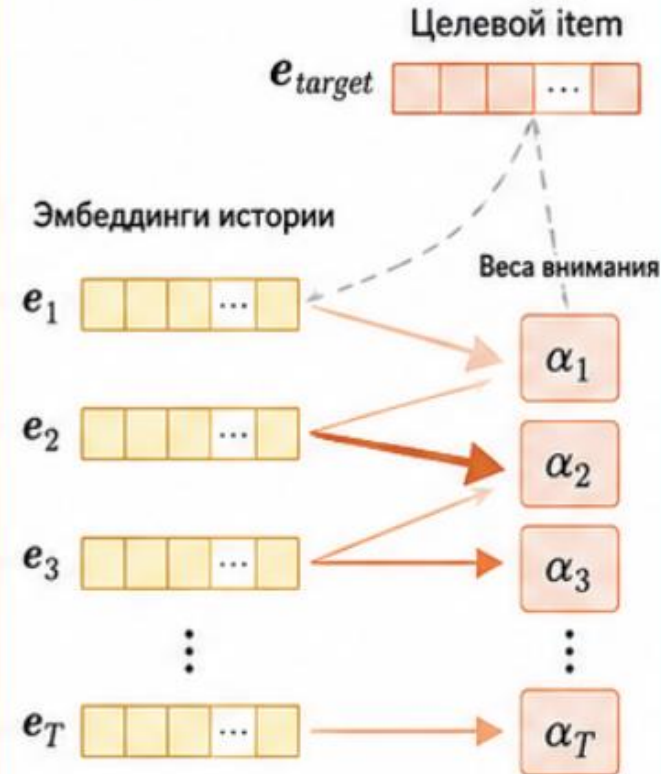
История пользователя — последовательность прошлых просмотренных / кликнутых items.



История (длина T)

Каждый просмотренный / кликнутый item получает embedding.

3. Local Activation Unit (attention к истории)



$\alpha_j = \text{Attention}(e_j, e_{\text{target}})$
например:
 $\text{MLP}([e_j, e_{\text{target}}, e_j - e_{\text{target}}, e_j \odot e_{\text{target}}])$

Чем релевантнее прошлый item для target item, тем больше вес α_j .

низкая релевантность → высокая релевантность

4. Агрегация интереса пользователя

Взвешенная сумма эмбедингов истории с весами внимания.

$$v_u(\text{target}) = \sum_{j=1}^T \alpha_j e_j$$

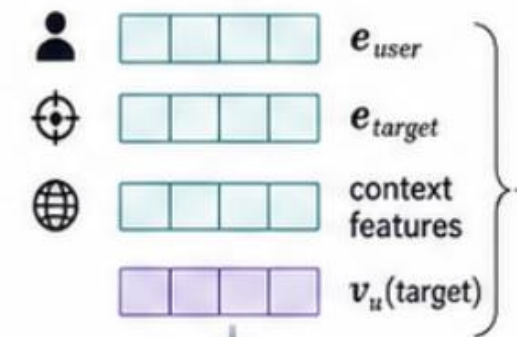
target-aware user interest



- одна и та же история дает разные интересы для разных target item
- DIN не использует одинаковый вес для всех прошлых действий

5. CTR-голова и выход

Объединяем признаки и интерес пользователя (target-aware).



Конкатенация

Dense(128) + ReLU

Dense(64) + ReLU

y (логит)

sigmoid

$$p(\text{click} = 1 | x) = \sigma(y)$$

Функция потерь
LogLoss / Binary Cross-Entropy

Метрики
• AUC
• LogLoss
• GAUC

Поток работы DIN



Ключевая идея DIN

- История пользователя важна.
- важность каждого прошлого item зависит от target item.
- attention помогает выделить релевантные интересы и улучшить CTR prediction.

openbenchmark.github.io/BARS/CTR/leaderboard/amazonelectronics_x1.html

W Tab

Все для Вас СМО...

Gmail

YouTube

Time-Aware Next-...

Time-Aware Next-...

c

BARS BENCHMARK

Welcome to BARS

PAPER LIST

CTR Prediction

Candidate Matching

CTR PREDICTION

Benchmark Overview

Benchmark Datasets

Benchmark Code

BARS-CTR Leaderboard

BARS-SeqCTR Leaderboard

AmazonElectronics_x1

KuaiVideo_x1

MicroVideo1.7M_x1

TaobaoAd_x1

CANDIDATE MATCHING

Benchmark Overview

Benchmark Datasets

Benchmark Code

BARS-Match Leaderboard

gAUC

AUC

Logloss

Show

10

entries

Search:

Rank	Year	Publication	Model	gAUC	AUC	Logloss	Running Steps	Contributor
1	2020	CIKM'20	DMIN	0.8856	0.8877	0.4246		Zhu et al.
2	2019	AAAI'19	DIEN	0.8856	0.8888	0.4257		Zhu et al.
3	2020	AAAI'20	DMR	0.8851	0.8873	0.4277		Zhu et al.
4	2019	DLP-KDD'19	BST	0.8841	0.8864	0.4301		Zhu et al.
5	2018	KDD'18	DIN	0.8835	0.8860	0.4302		Zhu et al.
6	2023	KDD'23	PPNet	0.8797	0.8817	0.4398		Zhu et al.
7	2022	NeurIPS'22	APG_DCNv2	0.8794	0.8820	0.4373		Zhu et al.
8	2021	KDD'21	AOANet	0.8791	0.8812	0.4399		Zhu et al.
9	2021	WWW'21	DCN-V2	0.8790	0.8812	0.4445		Zhu et al.
10	2018	KDD'18	xDeepFM	0.8790	0.8813	0.4387		Zhu et al.

Showing 1 to 10 of 16 entries

Previous

1

2

Next

FM

FmFM

DCN

AutoInt+

DeepFM

YoutubeDNN

xDeepFM

DCN-V2

AOANet

APG_DCNv2

PPNet

DIN

BST

DMR

DIEN

DMIN

0.42

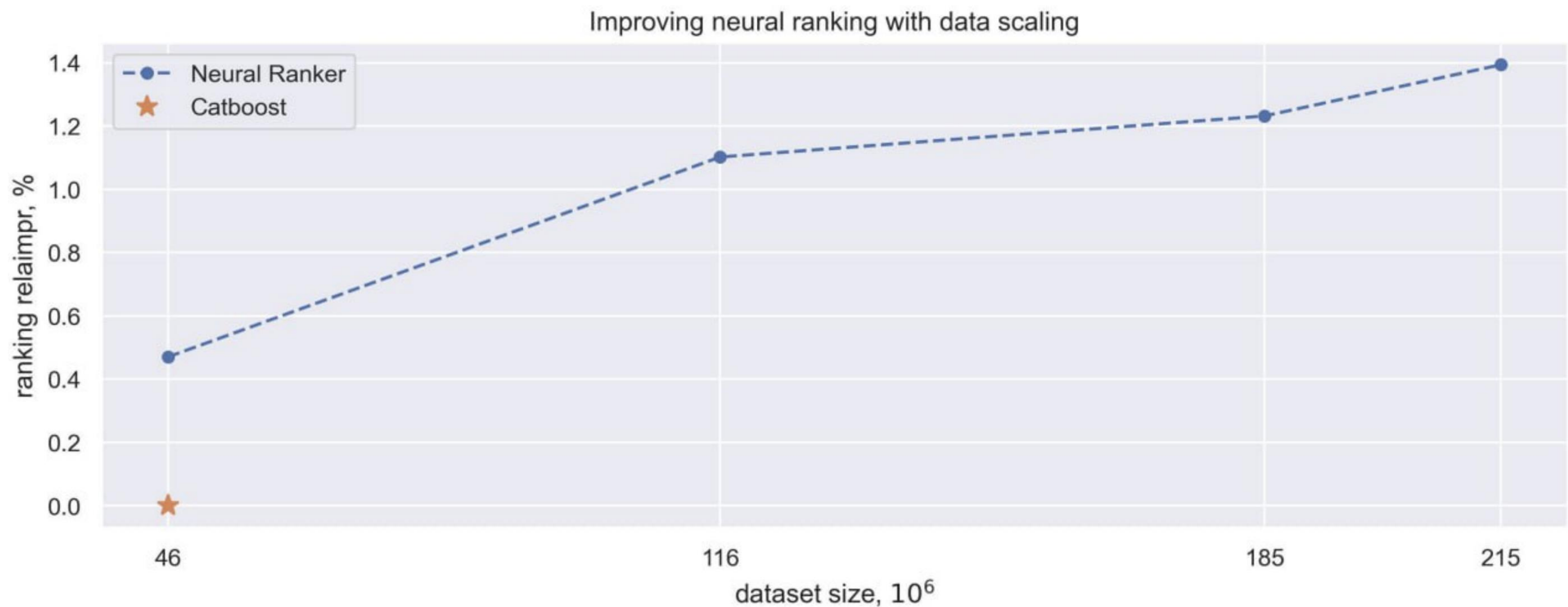
Законы масштабирования

Information Retriever

3 897 subscribers

Pinned message

Привет! Я (@kkhrylchenko).



Спасибо за
внимание